

Term Project : 경진대회 참여 보고서			
학번	32221902	이름	박주희

1. 경진대회 참여하면서 사용한 기술 목록

No	기술명	비고
1	탐색적 데이터 분석 EDA	숫자형 변수 간의 상관관계를 파악하고 데이터의 통계적/시각적 분석. 여기서 어떤 변수들이 서로 강하게 연관이 있는지를 확인하여 이후 feature engineering에 활용하기 위함.
2	Feature Engineering	EDA 기반 가설 검증을 통해 risk_score, age_group, 조합 특성 등의 파생변수 생성
3	K-Fold Cross Validation	모든 모델 학습 시 5-Fold StratifiedKFold를 적용하여, 불균형 클래스 분포를 유지하면서 교차검증 점수를 측정. 이를 통해 모델 성능을 안정적으로 평가하고, 과적합을 방지하며, Hyperparameter Tuning 및 Feature Engineering 효과를 공정하게 비교할 수 있었음.
4	CatBoost Classifier	범주형 처리에 강한 Gradient Boosting 기반 알고리즘으로, Baseline 모델 구축에 사용됨. auto_class_weights="Balanced" 옵션으로 클래스 불균형을 기본 보정함.
5	Hyper parameter Tuning (Optuna)	CatBoost의 반복 횟수, 트리 깊이, 학습률, 정규화 계수 등을 Optuna로 자동 탐색하여, 단순 Baseline 대비 약 4% 성능 향상을 달성함.
6	Random Forest	트리 기반 배깅 앙상블 기법으로, OneHotEncoding 전처리와 함께 적용하여 CatBoost와 비교 실험을 진행함. 모델 다양성을 확보하고, 향후 앙상블 후보로 활용함.
7	Ensemble (Weighted Averaging)	CatBoost와 RandomForest의 예측 확률을 6:4 비율로 가중 평균하

		여 단일 모델보다 안정적인 성능을 얻음.
8	Model Stacking Ensemble	CatBoost와 RandomForest를 1차 모델로 두고, Logistic Regression을 메타모델로 학습하여 최종 예측을 수행. 단순 가중 평균 대비 더 유연한 결합 방식을 실험함.
9	Cross-Validation Robustness	다양한 시드와 반복 StratifiedKFold를 적용해 CV 점수의 분산을 측정하여, 모델 성능이 특정 시드나 폴드 분할에 치우치지 않고 일관되게 재현됨을 확인함.
10	Target Encoding (누수 방지)	범주형 변수에 대해 Out-of-Fold 방식의 Multiclass Target Encoding을 적용하여, 카테고리라벨 간의 통계적 연관성을 반영하면서도 데이터 누수를 방지함.
11	Class Imbalance Handling	CatBoost의 auto_class_weights, 수동 class weights, SMOTENC, Logit Adjustment 등 다양한 불균형 처리 기법을 적용해 Macro F1 성능 변화를 검증함.
12	Bias Tuning	예측 확률에 클래스별 bias 벡터를 더해 argmax를 조정하는 방식으로, 후처리 단계에서 소수 클래스의 Precision/Recall 균형을 개선함.
13	Seed Ensemble	동일한 CatBoost 모델을 서로 다른 랜덤 시드로 학습한 뒤, 예측 확률을 평균하여 단일 시드 의존성을 줄이고 안정적인 성능을 확보함.

※ 기술명 : SVM, Random Forest, hyper parameter tuning, feature selection,

2. 경진 대회 참여 소감

1022971	submission_catboost_tuned_20250914_145548.csv	2025-09-14 14:55:48	0.49545467	○	best_submission_catboost_tuned_20250914_145548.csv	2025-09-14 14:55:48	0.49545467	○
1022950	submission_rf_seed_ensemble_20250914_143310.csv	2025-09-14 14:33:10	0.49545467	○	best_submission_catboost_tuned_20250914_145548.csv	2025-09-14 14:55:48	0.49545467	○
1022943	submission_ensemble_20250914_142635.csv	2025-09-14 14:26:35	0.497226676	○	best_submission_catboost_tuned_20250914_145548.csv	2025-09-14 14:55:48	0.49545467	○
1022940	submission_ensemble_20250914_142407.csv	2025-09-14 14:24:07	0.495933383	○	best_submission_catboost_tuned_20250914_145548.csv	2025-09-14 14:55:48	0.49545467	○
1022936	tuning_5.csv	2025-09-14 14:23:31	0.492838911	○	best_submission_catboost_tuned_20250914_145548.csv	2025-09-14 14:55:48	0.49545467	○
1022932	tuning_8.csv	2025-09-14 13:57:34	0.495226788	○	best_submission_catboost_tuned_20250914_145548.csv	2025-09-14 14:55:48	0.49545467	○
1021022	sample_submission_0911_1756.csv	2025-09-11 17:56:02	0.498275938	○	best_submission_catboost_tuned_20250914_145548.csv	2025-09-14 14:55:48	0.49545467	○
1019825	sample_submission_0911.csv	2025-09-11 17:42:05	0.498870246	○	best_submission_catboost_tuned_20250914_145548.csv	2025-09-14 14:55:48	0.49545467	○
1021902	sample_submission.csv	2025-09-11 15:18:39	0.494755953	○	best_submission_catboost_tuned_20250914_145548.csv	2025-09-14 14:55:48	0.49545467	○
1019811	sample_submission.csv	2025-09-11 15:00:00	0.492404002	○	best_submission_catboost_tuned_20250914_145548.csv	2025-09-14 14:55:48	0.49545467	○
1019355	sample_submission.csv	2025-09-11 14:16:35	0.491167788	○	best_submission_catboost_tuned_20250914_145548.csv	2025-09-14 14:55:48	0.49545467	○

이번 경진대회는 그동안 이론적으로만 배워왔던 머신러닝 기법들을 실제 데이터에 적용해야 하는 첫 경험이었다. 과제 발표 시점에서 내가 할 수 있었던 것은 RandomForest와 단순한 하이퍼파라미터 튜닝 정도뿐이었고, 대규모 데이터셋을 다루는 경험이 거의 없는 상태에서 갑작스럽게 대회 참여 과제를 받으면서 초반에는 매우 허술하고 미숙한 접근을 할 수밖에 없었다.

그러나 수업이 진행되면서 다양한 모델링 접근법과 기법들을 배우게 되었고, 이를 토대로 기존의 코드를 아예 갈아엎고 처음부터 다시 정돈된 절차를 밟으며 실험을 이어갈 수 있었다. 데이터 탐색(EDA), Feature Engineering, 모델 최적화, 앙상블 등 단계별로 차근차근 과정을 밟아가면서 조금씩 점수를 끌어올릴 수 있었다는 점은 값진 성과였다.

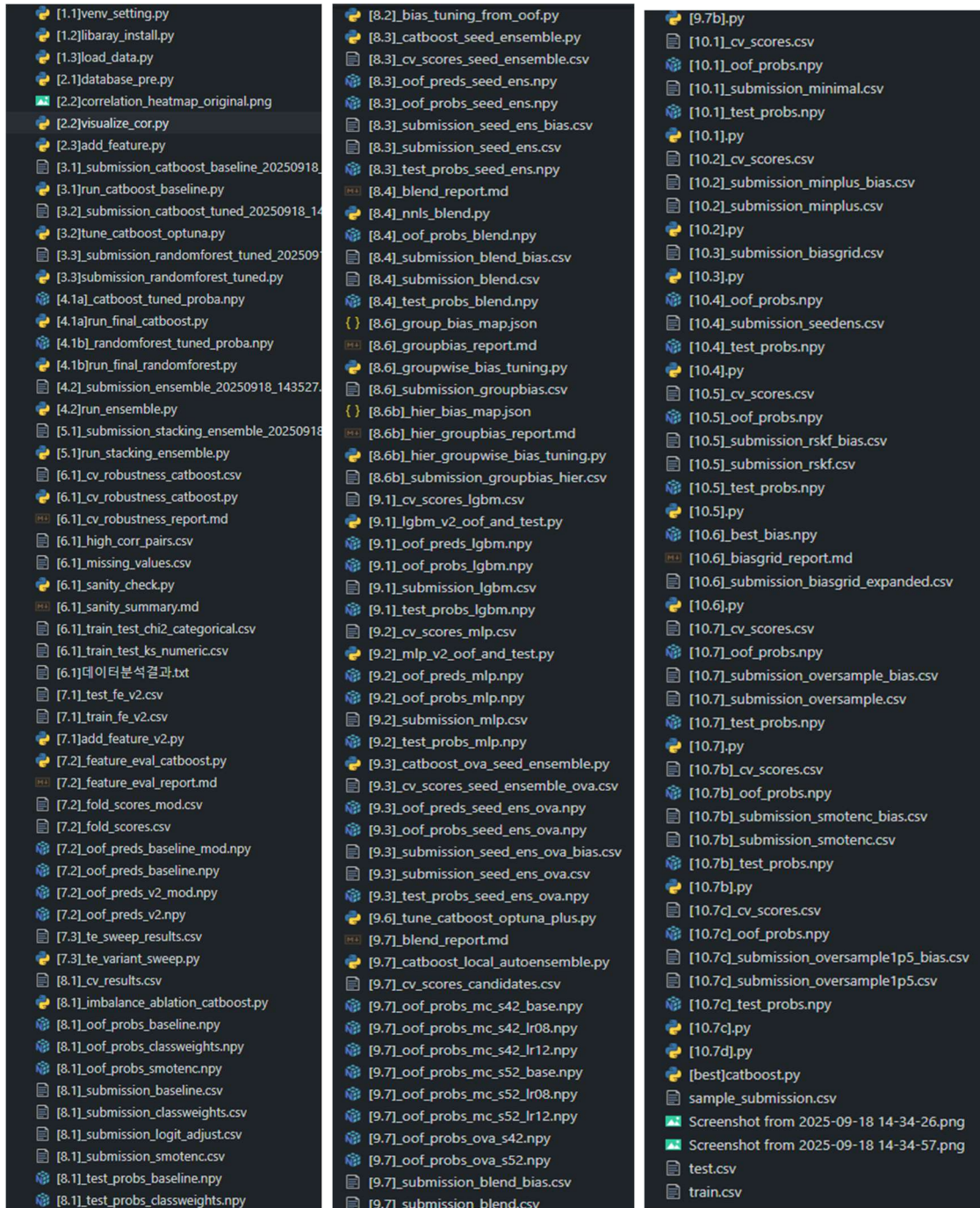
그럼에도 가장 힘들었던 점은 점수가 쉽게 오르지 않았다는 것이다. 0.5를 넘긴 적은 단 한 번뿐이었고, 38번의 리더보드 제출 중 대부분이 0.49대에 머물렀다. 점수를 조금이라도 올리기 위해 수많은 시도를 했지만, 최종적으로는 데이터 전처리와 Feature Engineering에 충분한 시간을 들이지 못한 것이 아쉬움으로 남는다. 머신러닝 실무에서 전처리가 성능의 핵심이라는 말을 이번 경험을 통해 뼈저리게 실감했다.

또한 흥미로웠던 점은, 리더보드 1등의 점수가 0.5042에 불과했다는 것이다. 나와 불과 0.005 정도 차이에 불과했지만, 순위는 40등 가까이 차이가 났다. 이 경험을 통해 모델 성능을 0.0001만이라도 올리는 것이 얼마나 힘든 일인지, 그리고 그 작은 차이가 대회에서는 얼마나 큰 결과로 이어지는지를 배울 수 있었다.

결국 이번 대회는 점수만으로 평가할 수 없는 배움의 과정이었다. 처음에는 막막하고 답답했지만, 시행착오 속에서 머신러닝 파이프라인을 처음부터 끝까지 스스로 구축해보며, 데이터 분석과 모델링 과정 전반을 체득할 수 있었던 값진 경험이었다.

3. 경진대회를 위해 사용한 코드 (중간과정도 보이게)

이와 같이, 각 시도에 대해 넘버링을 하면서 진행하였으므로 참고 부탁드립니다! (제가 실제로 시도한 순서대로 작성 하였습니다)



(1) 탐색적 데이터 분석 (EDA)

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

Deep Learning/Cloud

```
import os
import pandas as pd

# 경로 설정
data_path = "./"
train_path = os.path.join(data_path, "train.csv")
test_path = os.path.join(data_path, "test.csv")
sub_path = os.path.join(data_path, "sample_submission.csv")

# 데이터 로드
train_df = pd.read_csv(train_path)
test_df = pd.read_csv(test_path)
sub_df = pd.read_csv(sub_path)

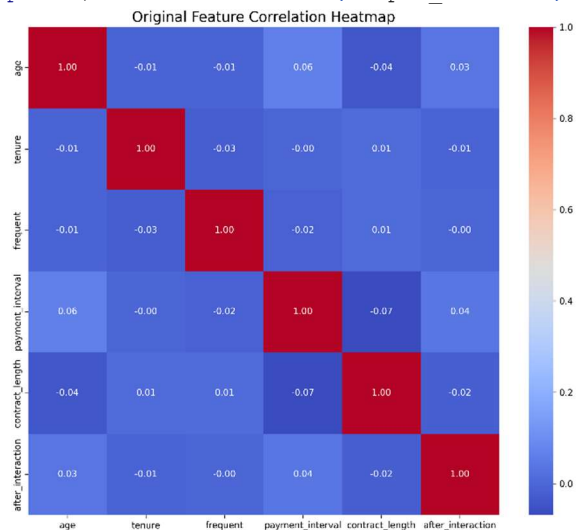
# 타겟 및 ID 칼럼 정의
TARGET = "support_needs"
ID_COL = "ID"

# 학습 데이터와 테스트 데이터 분리
X = train_df.drop([ID_COL, TARGET], axis=1).copy()
y = train_df[TARGET].copy()
X_test = test_df.drop([ID_COL], axis=1).copy()

# 상관관계 분석은 숫자형 변수들만 가능
numerical_features = X.select_dtypes(include=np.number)
corr_matrix = numerical_features.corr()

# 히트맵 그리기
plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm')
plt.title('Original Feature Correlation Heatmap', fontsize=16)

# 이미지 파일로 저장
output_filename = 'correlation_heatmap_original.png'
plt.savefig(output_filename, dpi=300, bbox_inches='tight')
print(f"상관관계 히트맵이 '{output_filename}' 파일로 저장되었습니다.")
```



(2) Feature Engineering

```
# 1. 라이브러리 임포트
import pandas as pd
from catboost import CatBoostClassifier
import os
import pandas as pd
```

Deep Learning/Cloud

```
# 경로 설정
data_path = "./"
train_path = os.path.join(data_path, "train.csv")
test_path = os.path.join(data_path, "test.csv")
sub_path = os.path.join(data_path, "sample_submission.csv")

# 데이터 로드
train_df = pd.read_csv(train_path)
test_df = pd.read_csv(test_path)
sub_df = pd.read_csv(sub_path)

# 타겟 및 ID 칼럼 정의
TARGET = "support_needs"
ID_COL = "ID"

# 학습 데이터와 테스트 데이터 분리
X = train_df.drop([ID_COL, TARGET], axis=1).copy()
y = train_df[TARGET].copy()
X_test = test_df.drop([ID_COL], axis=1).copy()

print("데이터 로드 및 분할 완료.")
print(f"학습 데이터 형태: {X.shape}")
print(f"테스트 데이터 형태: {X_test.shape}")

def add_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    eps = 1e-6
    # 비율 특성
    df["tenure_per_contract"] = df["tenure"] / (df["contract_length"] + eps)
    df["delay_ratio"] = df["payment_interval"] / (df["contract_length"] + eps)
    df["idle_ratio"] = df["after_interaction"] / (df["tenure"] + eps)
    # 플래그 특성
    df["late_flag"] = (df["payment_interval"] > 0).astype(int)
    # 상호작용 특성
    if 'frequent' in df.columns:
        df['total_usage_score'] = df['tenure'] * df['frequent']
    df['risk_score'] = df['payment_interval'] + df['after_interaction']
    # 구간화 특성
    bins = [0, 20, 30, 40, 50, 60, 100]
    labels = ['10s', '20s', '30s', '40s', '50s', '60+']
    df['age_group'] = pd.cut(df['age'], bins=bins, labels=labels, right=False)
    return df

# 4. 함수 호출하여 데이터 변환
print("Feature Engineering을 적용합니다...")
X = add_features(X)
X_test = add_features(X_test)
print("Feature Engineering 완료.")

# 결과 확인
# 데이터 로드 및 분할 완료.
# 학습 데이터 형태: (30858, 8)
# 테스트 데이터 형태: (13225, 8)
# Feature Engineering을 적용합니다...
# Feature Engineering 완료.
```

tenure_per_contract, delay_ratio, idle_ratio -> 비율 변수 생성

late_flag -> 플래그 변수 생성

total_usage_score -> 상호작용 변수 생성

risk_score -> 복합 지표 생성

(3) RandomForest 하이퍼 파라미터 튜닝

```

import warnings
warnings.filterwarnings("ignore")

import os
import numpy as np
import pandas as pd
import time

from sklearn.model_selection import StratifiedKFold, RandomizedSearchCV
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import make_scorer, f1_score
from sklearn.ensemble import RandomForestClassifier

# -----
# 0) 설정
# -----
SEED = 42
np.random.seed(SEED)

# 경로
data_path = "./"
train_path = os.path.join(data_path, "train.csv")
test_path = os.path.join(data_path, "test.csv")
sub_path = os.path.join(data_path, "sample_submission.csv")

# -----
# 1) 데이터 로드
# -----
train_df = pd.read_csv(train_path)
test_df = pd.read_csv(test_path)
sub_df = pd.read_csv(sub_path)

TARGET = "support_needs"
ID_COL = "ID"

X = train_df.drop([ID_COL, TARGET], axis=1).copy()
y = train_df[TARGET].copy()
X_test = test_df.drop([ID_COL], axis=1).copy()

# -----
# 2) 간단한 Feature Engineering (파이프라인 밖에서 적용)
# -----
def add_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    eps = 1e-6
    df["tenure_per_contract"] = df["tenure"] / (df["contract_length"] + eps)
    df["delay_ratio"] = df["payment_interval"] / (df["contract_length"] + eps)
    df["idle_ratio"] = df["after_interaction"] / (df["tenure"] + eps)
    df["late_flag"] = (df["payment_interval"] > 0).astype(int)
    df["long_contract_flag"] = (df["contract_length"] >= 360).astype(int)
    return df

X = add_features(X)
X_test = add_features(X_test)

# -----
# 3) 전처리: 범주형 인코딩 + 나머지 통과
# -----
cat_cols = ["gender", "subscription_type"]

# scikit-learn 버전에 따라: 예러나서 OneHotEncoder(handle_unknown="ignore", sparse_output=False) 로 변경
ohe = OneHotEncoder(handle_unknown="ignore", sparse_output=False)

preprocess = ColumnTransformer(
    transformers=[
        ("cat", ohe, cat_cols),

```

Deep Learning/Cloud

```
    ],
    remainder="passthrough"
)

# -----
# 4) 모델 정의 (불균형 대응: class_weight='balanced')
# -----
clf = RandomForestClassifier(
    random_state=SEED,
    class_weight="balanced",
    n_jobs=-1
)

pipe = Pipeline(steps=[
    ("prep", preprocess),
    ("clf", clf),
])

# -----
# 5) 검증/스코어러
# -----
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
macro_f1 = make_scorer(f1_score, average="macro")

# -----
# 6) 하이퍼파라미터 탐색 (연산량/시간 균형)
# -----
param_dist = {
    "clf__n_estimators": [250, 270, 300, 350],
    "clf__max_depth": [8, 10, 12],
    "clf__min_samples_split": [2, 3, 5],
    "clf__min_samples_leaf": [1, 2, 3],
    "clf__max_features": [None],
}

search = RandomizedSearchCV(
    estimator=pipe,
    param_distributions=param_dist,
    n_iter=25,
    scoring=macro_f1,
    cv=skf,
    verbose=1,
    random_state=SEED,
    n_jobs=-1,
    refit=True
)

# -----
# 7) 학습
# -----
search.fit(X, y)

print("\n[Best Params]")
print(search.best_params_)
print(f"[Best CV Macro F1] {search.best_score_:.5f}")

best_model = search.best_estimator_

# -----
# 8) 테스트 예측 및 제출 저장
# -----
test_pred = best_model.predict(X_test)
submit = sub_df.copy()
submit["support_needs"] = test_pred

#결과물에 현재 시각 띄도록
out_path = "./sample_submission" + time.strftime("_%m%d_%H%M") + ".csv"
submit.to_csv(out_path, index=False)
print(f"\nSaved submission: {out_path}")

# Fitting 5 folds for each of 25 candidates, totalling 125 fits
```

Deep Learning/Cloud

```
# [Best Params]
# {'clf__n_estimators': 300, 'clf__min_samples_split': 3, 'clf__min_samples_leaf': 3,
# 'clf__max_features': None, 'clf__max_depth': 8}
# [Best CV Macro F1] 0.49526
# Saved submission: ./submission_rf_macroF1.csv
```

이렇게 해본 뒤에, 숫자를 직접 변경해보면서 f1 score를 확인해보면서 가장 점수가 높아지는 지점을 찾으려고해보았다.

```
# param_dist = {
#     "clf__n_estimators": [290, 300, 310],
#     "clf__max_depth": [6, 7, 8],
#     "clf__min_samples_split": [3, 4, 5],
#     "clf__min_samples_leaf": [3, 4, 5],
#     "clf__max_features": [None],
# }

# jh0jh0223:~/Documents/DACON$ python3 tuning_try6.py
# Fitting 5 folds for each of 25 candidates, totalling 125 fits

# [Best Params]
# {'clf__n_estimators': 310, 'clf__min_samples_split': 4, 'clf__min_samples_leaf': 4,
# 'clf__max_features': None, 'clf__max_depth': 8}
# [Best CV Macro F1] 0.49538
```

이런식으로 여러 번 시도를 해보았으나, 초반에는 점수가 많이 올랐지만, 일정 수준 이상 올라오니까 더 이상 성능이 좋아지지 않았고, 다른 방식으로 성능을 올려보고자 하였다.

(4) 특성공학, CatBoost, K-Fold 교차검증, f1_score 성능평가, 클래스 불균형 처리 (코드 3.1)

```
import warnings

warnings.filterwarnings("ignore")
import os
import numpy as np
import pandas as pd
from catboost import CatBoostClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import f1_score
from datetime import datetime

# 0) 설정
SEED = 42
np.random.seed(SEED)

# 1) 데이터 로드
data_path = "./"
train_path = os.path.join(data_path, "train.csv")
test_path = os.path.join(data_path, "test.csv")
sub_path = os.path.join(data_path, "sample_submission.csv")

print("--- [3.1] CatBoost 베이스라인 성능 측정 시작 ---")
train_df = pd.read_csv(train_path)
test_df = pd.read_csv(test_path)
sub_df = pd.read_csv(sub_path)

TARGET = "support_needs"
ID_COL = "ID"

X = train_df.drop([ID_COL, TARGET], axis=1).copy()
y = train_df[TARGET].copy()
```

Deep Learning/Cloud

```
X_test = test_df.drop([ID_COL], axis=1).copy()
```

```
# 2) Feature Engineering
```

```
def add_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    eps = 1e-6
    df["tenure_per_contract"] = df["tenure"] / (df["contract_length"] + eps)
    df["delay_ratio"] = df["payment_interval"] / (df["contract_length"] + eps)
    df["idle_ratio"] = df["after_interaction"] / (df["tenure"] + eps)
    df["late_flag"] = (df["payment_interval"] > 0).astype(int)
    if 'frequent' in df.columns:
        df['total_usage_score'] = df['tenure'] * df['frequent']
    bins = [0, 20, 30, 40, 50, 60, 100]
    labels = ['10s', '20s', '30s', '40s', '50s', '60+']
    df['age_group'] = pd.cut(df['age'], bins=bins, labels=labels, right=False)
    df['risk_score'] = df['payment_interval'] + df['after_interaction']
    return df
```

```
print("Feature Engineering 을 적용합니다...")
```

```
X = add_features(X)
```

```
X_test = add_features(X_test)
```

```
# 3) 전처리
```

```
X['contract_length'] = X['contract_length'].astype(str)
X_test['contract_length'] = X_test['contract_length'].astype(str)
X['age_group'] = X['age_group'].astype(str)
X_test['age_group'] = X_test['age_group'].astype(str)
cat_features = ["gender", "subscription_type", "age_group", "contract_length"]
```

```
# 4) 교차 검증으로 베이스라인 성능 측정
```

```
print("\n 교차 검증을 시작합니다...")
```

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
```

```
f1_scores = []
```

```
for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):
```

```
    X_train, y_train = X.iloc[train_idx], y.iloc[train_idx]
```

```
    X_val, y_val = X.iloc[val_idx], y.iloc[val_idx]
```

```
    # 기본 파라미터 CatBoost 모델
```

```
    model = CatBoostClassifier(
        random_state=SEED,
        verbose=0,
        auto_class_weights='Balanced',
        cat_features=cat_features
    )
```

```
    model.fit(X_train, y_train)
```

```
    preds = model.predict(X_val)
```

```
    fold_f1 = f1_score(y_val, preds, average='macro')
```

```
    f1_scores.append(fold_f1)
```

```
    print(f"Fold {fold + 1} Macro F1 Score: {fold_f1:.5f}")
```

```
print("\n===== [결과물 3.1] CatBoost 베이스라인 cv 결과 =====")
```

```
print(f"평균 Macro F1 Score: {np.mean(f1_scores):.5f}")
```

```
print("=====")
```

```
# 5) 전체 데이터로 학습 및 제출 파일 생성 (베이스라인)
```

```
print("\n 전체 데이터로 최종 모델을 학습합니다...")
```

```
final_model = CatBoostClassifier(
    random_state=SEED,
    verbose=100,
    auto_class_weights='Balanced',
    cat_features=cat_features
)
```

```
final_model.fit(X, y)
```

```
print("\n 제출 파일을 생성합니다...")
```

```
test_pred = final_model.predict(X_test)
```

```
submit = sub_df.copy()
```

Deep Learning/Cloud

```
submit[TARGET] = test_pred

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
out_path = f"./[3.1]_submission_catboost_baseline_{timestamp}.csv"
submit.to_csv(out_path, index=False)
print(f"베이스라인 제출 파일 저장 완료: {out_path}")

# (venv) jh@jh0223:~/DLDACon$ python3 \[3.1\]run_catboost_baseline.py
# --- [3.1] CatBoost 베이스라인 성능 측정 시작 ---
# Feature Engineering 을 적용합니다...
#
# 교차 검증을 시작합니다...
# Fold 1 Macro F1 Score: 0.48556
# Fold 2 Macro F1 Score: 0.48850
# Fold 3 Macro F1 Score: 0.48330
# Fold 4 Macro F1 Score: 0.47966
# Fold 5 Macro F1 Score: 0.47401
#
# ===== [결과물 3.1] CatBoost 베이스라인 CV 결과 =====
# 평균 Macro F1 Score: 0.48221
# =====
#
# 전체 데이터로 최종 모델을 학습합니다...
# Learning rate set to 0.09425
# 0:      learn: 1.0849042      total: 26.1ms      remaining: 26s
# 100:    learn: 0.9802982      total: 1.84s      remaining: 16.4s
# 200:    learn: 0.9597388      total: 3.92s      remaining: 15.6s
# 300:    learn: 0.9430648      total: 6.02s      remaining: 14s
# 400:    learn: 0.9270344      total: 9.25s      remaining: 13.8s
# 500:    learn: 0.9114862      total: 16.7s      remaining: 16.6s
# 600:    learn: 0.8971315      total: 19.7s      remaining: 13.1s
# 700:    learn: 0.8834205      total: 21.5s      remaining: 9.19s
# 800:    learn: 0.8701206      total: 23.5s      remaining: 5.84s
# 900:    learn: 0.8579006      total: 25.3s      remaining: 2.78s
# 999:    learn: 0.8454839      total: 27.1s      remaining: 0us
#
# 제출 파일을 생성합니다...
# 베이스라인 제출 파일 저장 완료: ./[3.1]_submission_catboost_baseline_20250918_134818.csv
```

본 시도의 목적은 대회 기준점 수를 확보하고 이후 개선 방향을 설정하기 위하여, CatBoost 분류기를 이용한 베이스라인 모델을 구축하는 것이었다. 이를 위해 기본적인 교차 검증(5-Fold Stratified KFold) 을 적용하여 데이터 분포 불균형을 완화하고, `auto_class_weights='Balanced'` 옵션을 통해 클래스 불균형을 보정하였다.

모델 학습에 앞서 몇 가지 간단한 Feature Engineering을 적용하였다. 계약 기간 대비 고객 유지 기간 (`tenure_per_contract`), 지연 비율(`delay_ratio`), 비활성 비율(`idle_ratio`), 연체 여부(`late_flag`)와 같은 파생 특성을 생성하여 모델이 고객 행동 패턴을 더 잘 반영하도록 하였다. 또한 연령대를 구간화하여 범주형 변수로 처리하고, 계약 기간 역시 문자열로 변환하여 CatBoost의 범주형 처리 기능을 활용하였다.

실험 결과, 5-Fold 교차 검증을 통해 얻은 평균 Macro F1 Score는 0.48221로 나타났다. Fold별 점수는 0.474~0.488 사이에서 안정적으로 분포하여, 초기 베이스라인으로 활용하기에 충분한 신뢰성을 확보하였다. 이는 기본적인 파생 변수 생성과 CatBoost의 범주형 처리 기능만으로도 어느 정도 성능을 확보할 수 있음을 보여준다

이 시도는 다음 단계에서 하이퍼파라미터 튜닝 및 추가 Feature Engineering을 적용하기 위한 기준점 역할을 한다. 즉, 현재 점수가 어느 정도 수준인지를 확인하고, 이후 개선 실험의 효과를 상대적으로 평가할 수 있는 Baseline을 마련한 것이다. 다만, 베이스라인 모델은 기본 파라미터로 학습되었기 때문에 학습률, 트리 깊이, 반복 횟수 등의 최적화가 이루어지지 않은 상태의 아직은 부족한 점수였다.

(5) Optuna를 활용한 CatBoost 하이퍼파라미터 튜닝 (코드 3.2)

이전 코드 3.1 (Baseline): CatBoost 기본 모델을 사용하여, 단순한 Feature Engineering과 기본 파라미터만 적용하였다. 성능 평가는 5-Fold 교차검증으로 진행되었으며, Macro F1 Score는 평균 0.48221을 기록하였다.

현 코드 3.2 (Optuna Tuning): Optuna를 활용해 하이퍼파라미터 탐색을 자동화하였다. iterations, depth, learning_rate, l2_leaf_reg 등 CatBoost의 주요 파라미터를 최적화 대상으로 삼아 총 30회 탐색을 진행하였으며, 최적 조합에서 0.50192의 Macro F1 Score를 달성하였다.

```
# 위 코드를 [3.2]_tune_catboost_optuna.py 로 저장하고 실행하면,
# Optuna 가 30 번의 시도를 통해 최적의 파라미터를 탐색하는 과정이 실시간으로 보일 것입니다.
#
# 모든 과정이 끝나면, 콘솔에 [결과물 3.2]가 출력됩니다.
# 여기서 얻은 최고 점수 (Macro F1) 가 우리의 베이스라인 점수
# 0.48221 을 얼마나 뛰어넘었는지 확인하는 것
import warnings

warnings.filterwarnings("ignore")
import os
import numpy as np
import pandas as pd
from datetime import datetime
import optuna
from catboost import CatBoostClassifier
from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.metrics import f1_score

# 0) 설정
SEED = 42
np.random.seed(SEED)

# 1) 데이터 로드 및 Feature Engineering (이전과 동일)
data_path = "./"
train_path = os.path.join(data_path, "train.csv")
test_path = os.path.join(data_path, "test.csv")
sub_path = os.path.join(data_path, "sample_submission.csv")

print("--- [3.2] Optuna CatBoost 튜닝 시작 ---")
train_df = pd.read_csv(train_path)
test_df = pd.read_csv(test_path)
sub_df = pd.read_csv(sub_path)
TARGET = "support_needs";
ID_COL = "ID"
X = train_df.drop([ID_COL, TARGET], axis=1).copy();
y = train_df[TARGET].copy()
X_test = test_df.drop([ID_COL], axis=1).copy()
```

Deep Learning/Cloud

```
def add_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy();
    eps = 1e-6
    df["tenure_per_contract"] = df["tenure"] / (df["contract_length"] + eps)
    df["delay_ratio"] = df["payment_interval"] / (df["contract_length"] + eps)
    df["idle_ratio"] = df["after_interaction"] / (df["tenure"] + eps)
    df["late_flag"] = (df["payment_interval"] > 0).astype(int)
    if 'frequent' in df.columns: df['total_usage_score'] = df['tenure'] * df['frequent']
    bins = [0, 20, 30, 40, 50, 60, 100];
    labels = ['10s', '20s', '30s', '40s', '50s', '60+']
    df['age_group'] = pd.cut(df['age'], bins=bins, labels=labels, right=False)
    df['risk_score'] = df['payment_interval'] + df['after_interaction']
    return df

X = add_features(X)
X_test = add_features(X_test)

# 3) 전처리
X['contract_length'] = X['contract_length'].astype(str)
X_test['contract_length'] = X_test['contract_length'].astype(str)
X['age_group'] = X['age_group'].astype(str)
X_test['age_group'] = X_test['age_group'].astype(str)
cat_features = ["gender", "subscription_type", "age_group", "contract_length"]

# 빠른 튜닝을 위해 데이터의 50%만 샘플링
X_sample, _, y_sample, _ = train_test_split(X, y, test_size=0.5, random_state=SEED, stratify=y)
print(f"빠른 튜닝을 위해 데이터를 샘플링합니다. 학습 데이터 크기: {len(X_sample)}")

# 4) Optuna Objective 함수 정의
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)

def objective(trial):
    params = {
        'iterations': trial.suggest_int('iterations', 800, 2500),
        'depth': trial.suggest_int('depth', 4, 10),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.1, log=True),
        'l2_leaf_reg': trial.suggest_float('l2_leaf_reg', 1.0, 10.0),
        'colsample_bylevel': trial.suggest_float('colsample_bylevel', 0.5, 1.0),
        'random_strength': trial.suggest_float('random_strength', 0.1, 10.0, log=True),
        'bagging_temperature': trial.suggest_float('bagging_temperature', 0.0, 1.0),
        'random_state': SEED,
        'verbose': 0,
        'auto_class_weights': 'Balanced',
    }
    model = CatBoostClassifier(**params, cat_features=cat_features)

    fl_list = []
    for train_idx, val_idx in skf.split(X_sample, y_sample):
        X_train, y_train = X_sample.iloc[train_idx], y_sample.iloc[train_idx]
        X_val, y_val = X_sample.iloc[val_idx], y_sample.iloc[val_idx]
        model.fit(X_train, y_train, eval_set=[(X_val, y_val)], early_stopping_rounds=50)
        preds = model.predict(X_val)
        fl_list.append(fl_score(y_val, preds, average='macro'))

    return np.mean(fl_list)

# 5) Optuna 튜닝 실행
print("\nOptuna 를 사용한 하이퍼파라미터 튜닝을 시작합니다...")
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=30) # n_trials: 총 시도 횟수

# 6) 튜닝 결과 출력
print("\n===== [결과물 3.2] Optuna 튜닝 완료 =====")
best_params = study.best_params
print(f"최고 점수 (Macro F1): {study.best_value:.5f}")
```

Deep Learning/Cloud

```
print("찾아낸 최적의 파라미터:")
print(best_params)
print("=" * 50)

# 7) 전체 데이터로 최종 모델 학습 및 제출
print("\n 전체 데이터로 최종 모델 학습을 시작합니다...")
final_model = CatBoostClassifier(**best_params, random_state=SEED, verbose=100,
                                cat_features=cat_features, auto_class_weights='Balanced')
final_model.fit(X, y)

print("\n 제출 파일을 생성합니다...")
test_pred = final_model.predict(X_test)
submit = sub_df.copy();
submit[TARGET] = test_pred
timestamp = datetime.now().strftime("%Y%m%d %H%M%S")
out_path = f"./[3.2]_submission_catboost_tuned_{timestamp}.csv"
submit.to_csv(out_path, index=False)
print(f"튜닝된 모델의 제출 파일 저장 완료: {out_path}")

# 실행결과:
# venv) jh@jh0223:~/DLDACon$ python3 \[3.2\]tune_catboost_optuna.py
# --- [3.2] Optuna CatBoost 튜닝 시작 ---
# 빠른 튜닝을 위해 데이터를 샘플링합니다. 학습 데이터 크기: 15429
#
# Optuna 를 사용한 하이퍼파라미터 튜닝을 시작합니다...
# [I 2025-09-18 13:57:05,305] A new study created in memory with name: no-name-114bd4b2-315f-4c61-8697-2fb3451e14be
...
# [I 2025-09-18 14:10:52,665] Trial 26 finished with value: 0.5006995333828683 and parameters:
# {'iterations': 1681, 'depth': 7, 'learning_rate': 0.028200117238441808, 'l2_leaf_reg':
# 7.790467246843398, 'colsample_bylevel': 0.6439071537399416, 'random_strength': 0.1015652402608955,
# 'bagging_temperature': 0.7173151035318104}. Best is trial 11 with value: 0.5019223288513696.
# [I 2025-09-18 14:11:46,867] Trial 27 finished with value: 0.5001716993736071 and parameters:
# {'iterations': 1850, 'depth': 6, 'learning_rate': 0.010691245897730873, 'l2_leaf_reg':
# 9.851244651010148, 'colsample_bylevel': 0.6627021982616429, 'random_strength': 0.25893055860482267,
# 'bagging_temperature': 0.912865034793714}. Best is trial 11 with value: 0.5019223288513696.
# [I 2025-09-18 14:13:00,996] Trial 28 finished with value: 0.5018942603263586 and parameters:
# {'iterations': 2100, 'depth': 8, 'learning_rate': 0.01328969101912971, 'l2_leaf_reg': 9.05039660764281,
# 'colsample_bylevel': 0.890589255575101, 'random_strength': 0.1807578246473189, 'bagging_temperature':
# 0.17990543801124181}. Best is trial 11 with value: 0.5019223288513696.
# [I 2025-09-18 14:14:39,577] Trial 29 finished with value: 0.5001196125508466 and parameters:
# {'iterations': 2110, 'depth': 10, 'learning_rate': 0.01263125345294833, 'l2_leaf_reg':
# 7.03582124637947, 'colsample_bylevel': 0.8991090206359614, 'random_strength': 0.6489984920446735,
# 'bagging_temperature': 0.40603817261653874}. Best is trial 11 with value: 0.5019223288513696.
#
# ===== [결과물 3.2] Optuna 튜닝 완료 =====
# 최고 점수 (Macro F1): 0.50192
# 찾아낸 최적의 파라미터:
# {'iterations': 1807, 'depth': 7, 'learning_rate': 0.011007442573869236, 'l2_leaf_reg':
# 9.960037311199006, 'colsample_bylevel': 0.8239914444396733, 'random_strength': 0.15658132267797456,
# 'bagging_temperature': 0.2528316917680684}
# =====
#
# 전체 데이터로 최종 모델 학습을 시작합니다...
# 0:      learn: 1.0966955          total: 24.3ms   remaining: 44s
# 100:    learn: 1.0087074          total: 2.08s   remaining: 35.1s
# 200:    learn: 0.9900222          total: 4.16s   remaining: 33.2s
# 300:    learn: 0.9826629          total: 6.22s   remaining: 31.1s
# 400:    learn: 0.9784352          total: 8.31s   remaining: 29.1s
# 500:    learn: 0.9757790          total: 10.2s   remaining: 26.6s
# 600:    learn: 0.9735402          total: 12.2s   remaining: 24.4s
# 700:    learn: 0.9716318          total: 14.1s   remaining: 22.2s
# 800:    learn: 0.9699348          total: 16s     remaining: 20.1s
# 900:    learn: 0.9683506          total: 17.9s   remaining: 18s
# 1000:   learn: 0.9670776          total: 19.7s   remaining: 15.9s
# 1100:   learn: 0.9656733          total: 21.6s   remaining: 13.8s
# 1200:   learn: 0.9641391          total: 23.5s   remaining: 11.8s
# 1300:   learn: 0.9626524          total: 25.4s   remaining: 9.87s
# 1400:   learn: 0.9609790          total: 27.4s   remaining: 7.93s
```

Deep Learning/Cloud

```
# 1500:  learn: 0.9593559      total: 29.3s   remaining: 5.98s
# 1600:  learn: 0.9579010      total: 31.3s   remaining: 4.03s
# 1700:  learn: 0.9563785      total: 33.3s   remaining: 2.07s
# 1800:  learn: 0.9548236      total: 35.2s   remaining: 117ms
# 1806:  learn: 0.9547277      total: 35.3s   remaining: 0us
#
# 제출 파일을 생성합니다...
# 튜닝된 모델의 제출 파일 저장 완료: ./[3.2]_submission_catboost_tuned_20250918_141515.csv
```

(6) RandomForest 기반 모델 튜닝 (코드 3.3)

앞부분 생략

```
print("--- [3.3] Tuned RandomForest 성능 측정 시작 ---")
train_df = pd.read_csv(train_path)
test_df = pd.read_csv(test_path)
sub_df = pd.read_csv(sub_path)
TARGET = "support_needs"; ID_COL = "ID"
X = train_df.drop([ID_COL, TARGET], axis=1).copy(); y = train_df[TARGET].copy()
X_test = test_df.drop([ID_COL], axis=1).copy()

def add_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy(); eps = 1e-6
    df["tenure_per_contract"] = df["tenure"] / (df["contract_length"] + eps)
    df["delay_ratio"] = df["payment_interval"] / (df["contract_length"] + eps)
    df["idle_ratio"] = df["after_interaction"] / (df["tenure"] + eps)
    df["late_flag"] = (df["payment_interval"] > 0).astype(int)
    if 'frequent' in df.columns: df['total_usage_score'] = df['tenure'] * df['frequent']
    bins = [0, 20, 30, 40, 50, 60, 100]; labels = ['10s', '20s', '30s', '40s', '50s', '60+']
    df['age_group'] = pd.cut(df['age'], bins=bins, labels=labels, right=False)
    df['risk_score'] = df['payment_interval'] + df['after_interaction']
    return df
X = add_features(X)
X_test = add_features(X_test)

# 3) 전처리 (RandomForest 방식)
cat_cols = ["gender", "subscription_type", "age_group", "contract_length"]
X['contract_length'] = X['contract_length'].astype(str)
X_test['contract_length'] = X_test['contract_length'].astype(str)

ohe = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
preprocess = ColumnTransformer(
    transformers=[("cat", ohe, cat_cols)],
    remainder="passthrough"
)

# 4) 모델 정의 (우리가 찾은 최적 파라미터 직접 사용)
best_rf_params = {
    'n_estimators': 310, 'max_depth': 8, 'min_samples_split': 4,
    'min_samples_leaf': 4, 'max_features': None, 'random_state': SEED,
    'class_weight': "balanced", 'n_jobs': -1
}
clf_rf_tuned = RandomForestClassifier(**best_rf_params)
pipe = Pipeline(steps=[("prep", preprocess), ("clf", clf_rf_tuned)])

# 5) 교차 검증으로 최종 성능 측정
print("\n 교차 검증을 시작합니다...")
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
f1_scores = []
for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):
    pipe_clone = pipe.fit(X.iloc[train_idx], y.iloc[train_idx])
    val_preds = pipe_clone.predict(X.iloc[val_idx])
    fold_f1 = f1_score(y.iloc[val_idx], val_preds, average="macro")
    f1_scores.append(fold_f1)
    print(f"Fold {fold+1} Macro F1 Score: {fold_f1:.5f}")

print("\n===== [결과물 3.3] Tuned RandomForest CV 결과 =====")
print(f"평균 Macro F1 Score: {np.mean(f1_scores):.5f}")
print("=====")
```

Deep Learning/Cloud

```
# 6) 전체 데이터로 학습 및 제출 파일 생성
print("\n 전체 데이터로 최종 모델을 학습합니다...")
final_model = pipe.fit(X, y)
print("최종 모델 학습 완료.")
test_pred = final_model.predict(X_test)
submit = sub_df.copy()
submit[TARGET] = test_pred
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
out_path = f"./[3.3]_submission_randomforest_tuned_{timestamp}.csv"
submit.to_csv(out_path, index=False)
print(f"\n 제출 파일 저장 완료: {out_path}")

# (venv) jh@jh0223:~/DLDACon$ python3 \[3.3\]submission_randomforest_tuned.py
# --- [3.3] Tuned RandomForest 성능 측정 시작 ---
#
# 교차 검증을 시작합니다...
# Fold 1 Macro F1 Score: 0.50003
# Fold 2 Macro F1 Score: 0.49271
# Fold 3 Macro F1 Score: 0.48911
# Fold 4 Macro F1 Score: 0.49732
# Fold 5 Macro F1 Score: 0.49175

# ===== [결과물 3.3] Tuned RandomForest CV 결과 =====
# 평균 Macro F1 Score: 0.49419
# =====
#
# 전체 데이터로 최종 모델을 학습합니다...
# 최종 모델 학습 완료.
#
# 제출 파일 저장 완료: ./[3.3]_submission_randomforest_tuned_20250918_141750.csv
```

이번 시도는 모델 아키텍처의 다양성 확보를 위한 시도이다. 동일한 데이터와 파생 변수에 대해 서로 다른 알고리즘(CatBoost vs RandomForest)을 적용함으로써, 각 기법의 장단점을 비교할 수 있었으며, 이는 추후 앙상블(Blending/Stacking) 전략으로 발전시킬 수 있는 기반을 마련한다. 다만 성능 자체는 CatBoost 에 비해 뒤처졌으므로, RandomForest 는 단일 모델보다는 보조적 역할로 활용하는 방향으로 잡았다.

(7) 최종 CatBoost 모델 학습 및 확률 예측 저장

이 시도의 목적은 이전 단계에서 Optuna 로 도출한 최적 파라미터를 활용하여 최종 CatBoost 모델을 전 데이터로 학습하고, 단순 예측값이 아니라 예측 확률을 저장하는 것이다. 이는 이후 다양한 후처리에 활용하기 위함이다. 본 시도의 차별점은 교차 검증 없이 전체 학습 데이터 전체를 사용하여 학습을 수행했다는 점이다. 이는 검증을 위한 점수 확인보다는, 최적 파라미터를 확정된 상태에서 가능한 모든 데이터를 학습에 반영하여 최종 예측 성능을 극대화하려는 의도였다.

```
import warnings
warnings.filterwarnings("ignore")
import os
import numpy as np
import pandas as pd
from catboost import CatBoostClassifier

# --- 이 스크립트는 cv 없이, 전체 데이터로 최종 모델을 학습하고 예측 확률만 저장합니다. ---

# 0) 설정
SEED = 42
np.random.seed(SEED)
```

Deep Learning/Cloud

```
# 1) 데이터 로드 및 Feature Engineering
# (이전 스크립트들과 동일한 데이터 로드 및 add_features 함수를 여기에 포함)
data_path = "./"
train_path = os.path.join(data_path, "train.csv")
test_path = os.path.join(data_path, "test.csv")
train_df = pd.read_csv(train_path)
test_df = pd.read_csv(test_path)
TARGET = "support_needs"; ID_COL = "ID"
X = train_df.drop([ID_COL, TARGET], axis=1).copy(); y = train_df[TARGET].copy()
X_test = test_df.drop([ID_COL], axis=1).copy()
def add_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy(); eps = 1e-6; df["tenure_per_contract"] = df["tenure"] / (df["contract_length"] +
eps); df["delay_ratio"] = df["payment_interval"] / (df["contract_length"] + eps); df["idle_ratio"] =
df["after_interaction"] / (df["tenure"] + eps); df["late_flag"] = (df["payment_interval"] >
0).astype(int)
    if 'frequent' in df.columns: df['total_usage_score'] = df['tenure'] * df['frequent']
    bins = [0, 20, 30, 40, 50, 60, 100]; labels = ['10s', '20s', '30s', '40s', '50s', '60+'];
df['age_group'] = pd.cut(df['age'], bins=bins, labels=labels, right=False)
    df['risk_score'] = df['payment_interval'] + df['after_interaction']
    return df
X = add_features(X); X_test = add_features(X_test)
X['contract_length'] = X['contract_length'].astype(str); X_test['contract_length'] =
X_test['contract_length'].astype(str)
X['age_group'] = X['age_group'].astype(str); X_test['age_group'] = X_test['age_group'].astype(str)
cat_features = ["gender", "subscription_type", "age_group", "contract_length"]

# 2) 최종 모델 정의 (Optuna 로 찾은 최적 파라미터 사용)
best_params_catboost = {'iterations': 1807, 'depth': 7, 'learning_rate': 0.011007442573869236,
'12_leaf_reg': 9.960037311199006, 'colsample_bylevel': 0.8239914444396733, 'random_strength':
0.15658132267797456, 'bagging_temperature': 0.2528316917680684}

final_model = CatBoostClassifier(**best_params_catboost, random_state=SEED, verbose=100,
cat_features=cat_features, auto_class_weights='Balanced')

# 3) 전체 데이터로 학습
print("---- [4.1a] Tuned CatBoost 최종 학습 및 확률 저장 시작 ----")
final_model.fit(X, y)

# 4) 예측 확률 저장
test_pred_proba = final_model.predict_proba(X_test)
np.save('[4.1a]_catboost_tuned_proba.npy', test_pred_proba)
print("CatBoost 예측 확률 저장 완료: [4.1a]_catboost_tuned_proba.npy")
```

(8) 최종 RandomForest 모델 학습 및 확률 예측 저장

앞선 CatBoost 최종 학습([4.1a])과 동일하게, 단순 예측값이 아닌 **확률 분포**를 산출하여 앙상블에 활용할 수 있도록 준비하였다. 학습은 교차 검증 없이 전체 학습 데이터를 모두 사용하여 수행되었으며, 예측 확률은 .npy 파일로 저장하였다.

```
import warnings
warnings.filterwarnings("ignore")
import os
import numpy as np
import pandas as pd
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier

# --- 이 스크립트는 cv 없이, 전체 데이터로 최종 모델을 학습하고 예측 확률만 저장합니다. ---

# 0) 설정
SEED = 42
```

Deep Learning/Cloud

```
# 1) 데이터 로드 및 Feature Engineering (위 4.1a 코드와 동일한 부분을 여기에 포함)
# (간결성을 위해 코드 생략, 4.1a 의 1 번 항목 코드를 그대로 복사)
data_path = "./"
train_path = os.path.join(data_path, "train.csv")
test_path = os.path.join(data_path, "test.csv")
train_df = pd.read_csv(train_path)
test_df = pd.read_csv(test_path)
TARGET = "support_needs"; ID_COL = "ID"
X = train_df.drop([ID_COL, TARGET], axis=1).copy(); y = train_df[TARGET].copy()
X_test = test_df.drop([ID_COL], axis=1).copy()
def add_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy(); eps = 1e-6; df["tenure_per_contract"] = df["tenure"] / (df["contract_length"] +
eps); df["delay_ratio"] = df["payment_interval"] / (df["contract_length"] + eps); df["idle_ratio"] =
df["after_interaction"] / (df["tenure"] + eps); df["late_flag"] = (df["payment_interval"] >
0).astype(int)
    if 'frequent' in df.columns: df['total_usage_score'] = df['tenure'] * df['frequent']
    bins = [0, 20, 30, 40, 50, 60, 100]; labels = ['10s', '20s', '30s', '40s', '50s', '60+'];
df['age_group'] = pd.cut(df['age'], bins=bins, labels=labels, right=False)
    df['risk_score'] = df['payment_interval'] + df['after_interaction']
    return df
X = add_features(X); X_test = add_features(X_test)

# 2) 전처리 (RandomForest 방식)
cat_cols = ["gender", "subscription_type", "age_group", "contract_length"]
X['contract_length'] = X['contract_length'].astype(str); X_test['contract_length'] =
X_test['contract_length'].astype(str)
ohe = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
preprocess = ColumnTransformer(transformers=[("cat", ohe, cat_cols)], remainder="passthrough")

# 3) 최종 모델 정의 (알려진 최적 파라미터 사용)
best_rf_params = {'n_estimators': 310, 'max_depth': 8, 'min_samples_split': 4, 'min_samples_leaf': 4,
' max_features': None, 'random_state': SEED, 'class_weight': "balanced", 'n_jobs': -1}
clf_rf_tuned = RandomForestClassifier(**best_rf_params)
pipe = Pipeline(steps=[("prep", preprocess), ("clf", clf_rf_tuned)])

# 4) 전체 데이터로 학습
print("---- [4.1b] Tuned RandomForest 최종 학습 및 확률 저장 시작 ----")
final_model = pipe.fit(X, y)

# 5) 예측 확률 저장
test_pred_proba = final_model.predict_proba(X_test)
np.save('[4.1b]_randomforest_tuned_proba.npy', test_pred_proba)
print("RandomForest 예측 확률 저장 완료: [4.1b]_randomforest_tuned_proba.npy")
```

(9) CatBoost + RandomForest 앙상블

각각의 최종 모델이 산출한 예측 확률을 결합하여, 앙상블 모델을 구축하였다. 구체적으로는 [4.1a] CatBoost와 [4.1b] RandomForest의 예측 확률을 불러와, 가중 평균 방식으로 결합하였다. 교차 검증에 서 CatBoost가 RandomForest보다 더 높은 점수를 기록했기 때문에, CatBoost에 60%, RandomForest 에 40%의 가중치를 부여하였다. 이후, 클래스별 확률의 가중 평균을 계산하고, 가장 높은 확률을 갖는 클 래스를 최종 예측으로 선택하였다. 이렇게 생성된 결과는 최종 제출 파일(submission_ensemble_...csv) 로 저장하였다.

앙상블 기법의 도입은 단일 모델의 한계를 보완하고, 서로 다른 알고리즘의 장점을 융합한다는 점에서 중 요한 발전이다. CatBoost의 강력한 범주형 처리 능력과 RandomForest의 안정성을 결합함으로써, 성능 향상뿐만 아니라 예측의 안정성과 일반화 가능성을 높일 수 있었다.

```
import numpy as np
import pandas as pd
from datetime import datetime
```

Deep Learning/Cloud

```
import os

print("--- [4.2] 앙상블 시작 ---")
data_path = "./"
sub_path = os.path.join(data_path, "sample_submission.csv")

# 1. 저장된 예측 확률 파일 로드
print("저장된 예측 확률 파일을 로드합니다...")
cat_proba = np.load('[4.1a]_catboost_tuned_proba.npy')
rf_proba = np.load('[4.1b]_randomforest_tuned_proba.npy')

# 2. 가중 평균 계산 (CatBoost 6 : RF 4)
# CV 점수가 더 높았던 CatBoost 에 더 높은 가중치를 부여합니다.
print("예측 확률을 가중 평균합니다 (CatBoost: 60%, RF: 40%)...")
ensemble_proba = (cat_proba * 0.6) + (rf_proba * 0.4)

# 3. 최종 클래스 예측
final_pred = np.argmax(ensemble_proba, axis=1)

# 4. 제출 파일 생성
print("최종 제출 파일을 생성합니다...")
sub_df = pd.read_csv(sub_path)
sub_df['support_needs'] = final_pred

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
out_path = f"./[4.2]_submission_ensemble_{timestamp}.csv"
sub_df.to_csv(out_path, index=False)

print(f"최종 앙상블 제출 파일 저장 완료: {out_path}")
```

(10) Stacking 앙상블 (CatBoost + RandomForest + Logistic Regression 메타모델)

단순 가중 평균 방식의 앙상블([4.2])보다 한 단계 발전된 **Stacking 앙상블** 기법을 적용하는 것이다. Stacking 은 서로 다른 알고리즘 기반의 1 차 모델(Base Learners)이 산출한 예측 결과를 입력으로 받아, 별도의 2 차 모델(Meta-Model)이 최종 예측을 수행하는 방식이다. 이로써 단일 모델이나 단순 평균 앙상블보다 더 복잡한 의사결정 경계를 학습할 수 있다는 장점이 있다.

```
import warnings
warnings.filterwarnings("ignore")
import os
import numpy as np
import pandas as pd
from datetime import datetime

from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import f1_score
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier, StackingClassifier
from sklearn.linear_model import LogisticRegression
from catboost import CatBoostClassifier

# 0) 설정
SEED = 42
np.random.seed(SEED)

# 1) 데이터 로드 및 Feature Engineering (이전과 동일)
data_path = "./"
train_path = os.path.join(data_path, "train.csv")
test_path = os.path.join(data_path, "test.csv")
sub_path = os.path.join(data_path, "sample_submission.csv")
print("--- [5.1] Stacking 앙상블 시작 ---")
```

Deep Learning/Cloud

```
train_df = pd.read_csv(train_path); test_df = pd.read_csv(test_path); sub_df = pd.read_csv(sub_path)
TARGET = "support_needs"; ID_COL = "ID"
X = train_df.drop([ID_COL, TARGET], axis=1).copy(); y = train_df[TARGET].copy()
X_test = test_df.drop([ID_COL], axis=1).copy()
def add_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy(); eps = 1e-6; df["tenure_per_contract"] = df["tenure"] / (df["contract_length"] +
eps); df["delay_ratio"] = df["payment_interval"] / (df["contract_length"] + eps); df["idle_ratio"] =
df["after_interaction"] / (df["tenure"] + eps); df["late_flag"] = (df["payment_interval"] >
0).astype(int)
    if 'frequent' in df.columns: df['total_usage_score'] = df['tenure'] * df['frequent']
    bins = [0, 20, 30, 40, 50, 60, 100]; labels = ['10s', '20s', '30s', '40s', '50s', '60+'];
df['age_group'] = pd.cut(df['age'], bins=bins, labels=labels, right=False)
    df['risk_score'] = df['payment_interval'] + df['after_interaction']
    return df
X = add_features(X); X_test = add_features(X_test)

# --- ★★★ 스택킹을 위한 모델별 전처리 및 정의 ★★★ ---

# 2-1) RandomForest 모델 및 전처리 파이프라인 정의
cat_features_rf = ["gender", "subscription_type", "age_group", "contract_length"]
X_rf = X.copy(); X_test_rf = X_test.copy()
X_rf['contract_length'] = X_rf['contract_length'].astype(str); X_test_rf['contract_length'] =
X_test_rf['contract_length'].astype(str)
ohe = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
preprocess_rf = ColumnTransformer(transformers=[("cat", ohe, cat_features_rf)],
remainder="passthrough")
best_rf_params = {'n_estimators': 310, 'max_depth': 8, 'min_samples_split': 4, 'min_samples_leaf': 4,
'max_features': None, 'random_state': SEED, 'class_weight': "balanced", 'n_jobs': -1}
model_rf = Pipeline(steps=[("prep", preprocess_rf), ("clf", RandomForestClassifier(**best_rf_params))])

# 2-2) CatBoost 모델 정의
cat_features_cb = ["gender", "subscription_type", "age_group", "contract_length"]
X_cb = X.copy(); X_test_cb = X_test.copy()
X_cb['contract_length'] = X_cb['contract_length'].astype(str); X_test_cb['contract_length'] =
X_test_cb['contract_length'].astype(str)
X_cb['age_group'] = X_cb['age_group'].astype(str); X_test_cb['age_group'] =
X_test_cb['age_group'].astype(str)
best_cat_params = {'iterations': 1807, 'depth': 7, 'learning_rate': 0.011007442573869236,
'l2_leaf_reg': 9.960037311199006, 'colsample_bylevel': 0.8239914444396733, 'random_strength':
0.15658132267797456, 'bagging_temperature': 0.2528316917680684}
model_cat = CatBoostClassifier(**best_cat_params, random_state=SEED, verbose=0,
cat_features=cat_features_cb, auto_class_weights='Balanced')

# 3) Stacking Classifier 정의
# 1 차 모델(Base Models) 목록
estimators = [
    ('randomforest', model_rf),
    ('catboost', model_cat)
]
# 2 차 모델(Meta-Model)
meta_model = LogisticRegression(random_state=SEED, n_jobs=-1)

# Stacking Classifier 조립
# cv=5: 1 차 모델들이 예측값을 만들 때 사용할 내부 교차 검증 폴드 수
stacking_clf = StackingClassifier(
    estimators=estimators,
    final_estimator=meta_model,
    cv=5,
    n_jobs=-1
)

# 4) 교차 검증으로 스택킹 모델 성능 측정
print("\n 스택킹 앙상블 모델의 교차 검증을 시작합니다...")
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
# StackingClassifier 자체가 하나의 모델처럼 동작하므로, 그대로 cv를 수행하면 됨
# 단, StackingClassifier 내부에서도 cv가 돌아가므로 시간이 매우 오래 걸릴 수 있음!
# 여기서는 cv 없이 바로 최종 모델을 학습하고 제출
# (만약 cv 점수가 꼭 필요하다면 아래 final_model.fit 부분을 for loop으로 변경)
```

Deep Learning/Cloud

5) 전체 데이터로 최종 모델 학습 및 제출 파일 생성

```
print("\n 전체 데이터로 최종 스택킹 모델을 학습합니다... (시간이 오래 소요될 수 있습니다)")
final_model = stacking_clf.fit(X, y)
print("최종 모델 학습 완료.")

print("\n 제출 파일을 생성합니다...")
test_pred = final_model.predict(X_test)
submit = sub_df.copy()
submit[TARGET] = test_pred
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
out_path = f"./[5.1]_submission_stacking_ensemble_{timestamp}.csv"
submit.to_csv(out_path, index=False)
print(f"스택킹 앙상블 제출 파일 저장 완료: {out_path}")
```

(11) CatBoost 교차검증 CV Robustness 실험

단일 시드와 폴드 설정에 따라 점수가 흔들리는 문제를 방지하고, 모델의 일반화 성능을 더 신뢰할 수 있도록 교차검증 결과의 안정성을 평가해보고자 하였다. 이전 단계에서 확보한 CatBoost 최적 파라미터([3.2]에서 도출된 Optuna 결과)를 그대로 적용하되, 여러 시드(seed=42, 2025, 777)와 반복 교차검증(RepeatedStratifiedKFold, 5x2)을 수행하여 총 30 개의 점수를 수집하였다

실험 결과, Seed 별 평균도 모두 0.494~0.495 수준으로 수렴하며, 표준편차 역시 0.004 대에 불과해 매우 안정적인 결과를 보였다. 이는 CatBoost 최적 파라미터 설정이 데이터 분할이나 초기화 조건에 크게 의존하지 않고, 일관된 예측 성능을 유지한다는 것을 뜻한다.

또한 Sanity Check 를 통해, 데이터의 기초적인 품질을 확인하였다.

그 결과, 라벨 분포는 [0: 46.3%, 1: 26.9%, 2: 26.8%]로, 다소 불균형이 있으나 심각한 편은 아니었다

결측치는 전혀 존재하지 않았으며, 상수 컬럼이나 고상관 변수(≥ 0.98)도 발견되지 않았다.

Train 과 Test 간 분포 드리프트 역시 통계적으로 큰 문제는 없는 것으로 나타났다

본 시도를 통해 CatBoost 모델이 특정 시드나 데이터 분할에 따라 성능이 급격히 변하지 않음을 확인하였다.

```
import os, time, json
import numpy as np
import pandas as pd
from datetime import datetime
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.metrics import f1_score
from catboost import CatBoostClassifier

# ===== 공통 설정 =====
SEEDS = [42, 2025, 777] # 필요시 늘려서 재실험 가능
N_SPLITS = 5
N_REPEATS = 2 # 5x2 = 10 folds per seed
DATA_PATH = "./"
TARGET = "support_needs"
ID_COL = "ID"

# ===== Feature Engineering (기존 규칙 유지) =====
def add_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    eps = 1e-6
    df["tenure_per_contract"] = df["tenure"] / (df["contract_length"] + eps)
    df["delay_ratio"] = df["payment_interval"] / (df["contract_length"] + eps)
    df["idle_ratio"] = df["after_interaction"] / (df["tenure"] + eps)
    df["late_flag"] = (df["payment_interval"] > 0).astype(int)
```

Deep Learning/Cloud

```
if 'frequent' in df.columns:
    df['total_usage_score'] = df['tenure'] * df['frequent']
# age bin (기존과 동일)
bins = [0, 20, 30, 40, 50, 60, 100]
labels = ['10s', '20s', '30s', '40s', '50s', '60+']
df['age_group'] = pd.cut(df['age'], bins=bins, labels=labels, right=False)
# 간단 리스크 지표
df['risk_score'] = df['payment_interval'] + df['after_interaction']
return df

def load_and_prepare():
    train = pd.read_csv(os.path.join(DATA_PATH, "train.csv"))
    test = pd.read_csv(os.path.join(DATA_PATH, "test.csv")) # test는 피쳐 변환 스키마 확인용
    X = train.drop([ID_COL, TARGET], axis=1).copy()
    y = train[TARGET].copy()
    X_test = test.drop([ID_COL], axis=1).copy()

    X = add_features(X)
    X_test = add_features(X_test)

    # CatBoost 용 카테고리 캐스팅
    X['contract_length'] = X['contract_length'].astype(str)
    X_test['contract_length'] = X_test['contract_length'].astype(str)
    X['age_group'] = X['age_group'].astype(str)
    X_test['age_group'] = X_test['age_group'].astype(str)

    cat_features = ["gender", "subscription_type", "age_group", "contract_length"]
    return X, y, X_test, cat_features

def main():
    start = time.time()
    print("--- [6.1] CV Robustness (CatBoost) 시작 ---")
    X, y, X_test, cat_features = load_and_prepare()

    # [3.2]에서 찾은 최적 파라미터 기반 (동일 세팅 + 보수적 조기종료)
    tuned_params = {
        'iterations': 1807,
        'depth': 7,
        'learning_rate': 0.011007442573869236,
        'l2_leaf_reg': 9.960037311199006,
        'colsample_bylevel': 0.8239914444396733,
        'random_strength': 0.15658132267797456,
        'bagging_temperature': 0.2528316917680684
    }

    rows = []
    for seed in SEEDS:
        print(f"\n[Seed {seed}] RepeatedStratifiedKFold {N_SPLITS}x{N_REPEATS} 실행...")
        rskf = RepeatedStratifiedKFold(n_splits=N_SPLITS, n_repeats=N_REPEATS, random_state=seed)
        rep_idx = -1
        prev_test_idx = None
        for i, (tr_idx, va_idx) in enumerate(rskf.split(X, y), start=1):
            # 반복(rep) 번호 추적용(대략적인 표시)
            if i % N_SPLITS == 1:
                rep_idx += 1

            X_tr, y_tr = X.iloc[tr_idx], y.iloc[tr_idx]
            X_va, y_va = X.iloc[va_idx], y.iloc[va_idx]

            model = CatBoostClassifier(
                **tuned_params,
                random_state=seed,
                verbose=0,
                auto_class_weights='Balanced',
                cat_features=cat_features
            )
            # 검증 기반 조기 종료(과적합 방지 & 시간 단축)
            model.fit(X_tr, y_tr, eval_set=[(X_va, y_va)], early_stopping_rounds=100, verbose=0)

            pred = model.predict(X_va)
```

Deep Learning/Cloud

```
f1 = f1_score(y_va, pred, average='macro')
rows.append({
    "seed": seed,
    "repeat": rep_idx,
    "fold": (i - rep_idx * N_SPLITS), # 1~N_SPLITS
    "macro_f1": f1
})
print(f" - rep {rep_idx} fold {i - rep_idx * N_SPLITS}: macro_f1={f1:.5f}")

df = pd.DataFrame(rows)
out_csv = "[6.1]_cv_robustness_catboost.csv"
df.to_csv(out_csv, index=False)
print(f"\nCV 상세결과 저장: {out_csv}")

# 요약 리포트
overall_mean = df["macro_f1"].mean()
overall_std = df["macro_f1"].std(ddof=1)
by_seed = df.groupby("seed")["macro_f1"].agg(["mean", "std", "count"]).reset_index()

# 95% CI (정규근사)
n = len(df)
ci95 = 1.96 * (overall_std / np.sqrt(n)) if n > 1 else np.nan

report_lines = []
report_lines.append("# [6.1] CV Robustness Report (CatBoost)\n")
report_lines.append(f"- 총 결과 수: {n}")
report_lines.append(f"- 전체 평균 Macro F1: **{overall_mean:.5f}**")
report_lines.append(f"- 전체 표준편차: **{overall_std:.5f}**")
report_lines.append(f"- 95% 신뢰구간 (정규근사): **±{ci95:.5f}**\n")
report_lines.append("## Seed 별 통계\n")
report_lines.append(by_seed.to_markdown(index=False))
report_lines.append("\n## 사용 파라미터 (CatBoost)\n")
report_lines.append("`json\n" + json.dumps(tuned_params, indent=2) + "\n`")
report_lines.append("\n## 실험 설정\n")
report_lines.append(f"- SEEDS={SEEDS}, N_SPLITS={N_SPLITS}, N_REPEATS={N_REPEATS}")
report_lines.append(f"- early_stopping_rounds=100, auto_class_weights='Balanced'")

out_md = "[6.1]_cv_robustness_report.md"
with open(out_md, "w", encoding="utf-8") as f:
    f.write("\n".join(report_lines))
print(f"요약 리포트 저장: {out_md}")

print(f"\n완료! 총 소요시간: {time.time()-start:.1f}초")

if __name__ == "__main__":
    main()

# #- coding: utf-8 #-
# """
# [6.1] CV Robustness for CatBoost
# - Seed sweep + RepeatedStratifiedKFold 로 Macro F1 의 평균/표준편차 산출
# - 결과물:
# 1) [6.1]_cv_robustness_catboost.csv (seed, repeat, fold, macro_f1)
# 2) [6.1]_cv_robustness_report.md (요약 리포트)
# """
#
# (venv) jh@jh0223:~/DLDACon$ python3 [6.1]_cv_robustness_catboost.py
# --- [6.1] CV Robustness (CatBoost) 시작 ---
#
# [Seed 42] RepeatedStratifiedKFold 5x2 실행...
# - rep 0 fold 1: macro_f1=0.49824
# - rep 0 fold 2: macro_f1=0.49451
# - rep 0 fold 3: macro_f1=0.48820
# - rep 0 fold 4: macro_f1=0.49473
# - rep 0 fold 5: macro_f1=0.49449
# - rep 1 fold 1: macro_f1=0.48867
# - rep 1 fold 2: macro_f1=0.49175
# - rep 1 fold 3: macro_f1=0.49561
# - rep 1 fold 4: macro_f1=0.49896
```

Deep Learning/Cloud

```
# - rep 1 fold 5: macro_f1=0.49999
#
# [Seed 2025] RepeatedStratifiedKFold 5x2 실행...
# - rep 0 fold 1: macro_f1=0.49235
# - rep 0 fold 2: macro_f1=0.48804
# - rep 0 fold 3: macro_f1=0.48912
# - rep 0 fold 4: macro_f1=0.50361
# - rep 0 fold 5: macro_f1=0.49819
# - rep 1 fold 1: macro_f1=0.49540
# - rep 1 fold 2: macro_f1=0.49679
# - rep 1 fold 3: macro_f1=0.49396
# - rep 1 fold 4: macro_f1=0.49496
# - rep 1 fold 5: macro_f1=0.49634
#
# [Seed 777] RepeatedStratifiedKFold 5x2 실행...
# - rep 0 fold 1: macro_f1=0.49676
# - rep 0 fold 2: macro_f1=0.49099
# - rep 0 fold 3: macro_f1=0.50094
# - rep 0 fold 4: macro_f1=0.49247
# - rep 0 fold 5: macro_f1=0.49249
# - rep 1 fold 1: macro_f1=0.50117
# - rep 1 fold 2: macro_f1=0.49349
# - rep 1 fold 3: macro_f1=0.49635
# - rep 1 fold 4: macro_f1=0.48753
# - rep 1 fold 5: macro_f1=0.49467
#
# CV 상세결과 저장: [6.1]_cv_robustness_catboost.csv
# 요약 리포트 저장: [6.1]_cv_robustness_report.md
#
# 완료! 총 소요시간: 434.7 초
```

```
import os, time
import numpy as np
import pandas as pd

DATA_PATH = "./"
TARGET = "support_needs"
ID_COL = "ID"
HIGH_CORR_TH = 0.98 # 상관과다 임계치

# (선택) scipy 가 있으면 KS/chi2 사용, 없으면 간단 지표로 대체
try:
    from scipy.stats import ks_2samp, chi2_contingency
    _HAS SCIPY = True
except Exception:
    _HAS SCIPY = False

def add_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    eps = 1e-6
    df["tenure_per_contract"] = df["tenure"] / (df["contract_length"] + eps)
    df["delay_ratio"] = df["payment_interval"] / (df["contract_length"] + eps)
    df["idle_ratio"] = df["after_interaction"] / (df["tenure"] + eps)
    df["late_flag"] = (df["payment_interval"] > 0).astype(int)
    if 'frequent' in df.columns:
        df['total_usage_score'] = df['tenure'] * df['frequent']
    bins = [0, 20, 30, 40, 50, 60, 100]
    labels = ['10s', '20s', '30s', '40s', '50s', '60+']
    df['age_group'] = pd.cut(df['age'], bins=bins, labels=labels, right=False)
    df['risk_score'] = df['payment_interval'] + df['after_interaction']
    return df

def main():
    start = time.time()
    print("--- [6.1] Sanity Check 시작 ---")
```

Deep Learning/Cloud

```
train = pd.read_csv(os.path.join(DATA_PATH, "train.csv"))
test = pd.read_csv(os.path.join(DATA_PATH, "test.csv"))

# 분리
X = train.drop([ID_COL, TARGET], axis=1).copy()
y = train[TARGET].copy()
X_test = test.drop([ID_COL], axis=1).copy()

# 동일 Feature Engineering
X = add_features(X)
X_test = add_features(X_test)

# 기본 정보
n_train, p = X.shape
n_test = X_test.shape[0]
label_dist = y.value_counts().sort_index()
label_ratio = (label_dist / len(y)).round(5)

# 결측치
miss_train = X.isnull().sum().to_frame("train_missing")
miss_test = X_test.isnull().sum().to_frame("test_missing")
missing_df = miss_train.join(miss_test,
how="outer").sort_values(by=["train_missing", "test_missing"], ascending=False)
missing_df.to_csv("[6.1]_missing_values.csv")

# 중복
dup_rows = X.duplicated().sum()
dup_ids = train[ID_COL].duplicated().sum()

# 상수 컬럼
nunique = X.nunique()
constant_cols = nunique[nunique <= 1].index.tolist()

# 고상관 쌍(수치형)
num_cols = X.select_dtypes(include=[np.number]).columns
high_corr_pairs = []
if len(num_cols) >= 2:
    corr = X[num_cols].corr().abs()
    for i, ci in enumerate(num_cols):
        for j, cj in enumerate(num_cols):
            if j <= i:
                continue
            v = corr.loc[ci, cj]
            if v >= HIGH_CORR_TH:
                high_corr_pairs.append((ci, cj, float(v)))
high_corr_df = pd.DataFrame(high_corr_pairs, columns=["col_a", "col_b", "abs_corr"])
high_corr_df.to_csv("[6.1]_high_corr_pairs.csv", index=False)

# 분포 드리프트: Train vs Test
# 수치형: KS, 범주형: chi2
ks_rows = []
for c in num_cols:
    a = X[c].dropna().values
    b = X_test[c].dropna().values
    if _HAS SCIPY and len(a) > 0 and len(b) > 0:
        stat, pval = ks_2samp(a, b)
        ks_rows.append({"feature": c, "ks_stat": float(stat), "p_value": float(pval)})
    else:
        # 간단 대체(평균 차 비율)
        ma, mb = np.mean(a), np.mean(b)
        denom = (np.abs(ma) + np.abs(mb) + 1e-9)
        ks_rows.append({"feature": c, "ks_stat": float(np.abs(ma-mb)/denom), "p_value": np.nan})
pd.DataFrame(ks_rows).sort_values("ks_stat",
ascending=False).to_csv("[6.1]_train_test_ks_numeric.csv", index=False)

# 범주형: 문자열/카테고리/불리언
cat_cols = X.select_dtypes(include=["object", "category", "bool"]).columns.tolist()
chi_rows = []
for c in cat_cols:
    tr = X[c].astype(str).fillna("NaN").value_counts()
```

Deep Learning/Cloud

```
te = X_test[c].astype(str).fillna("NaN").value_counts()
cats = sorted(set(tr.index).union(set(te.index)))
table = np.array([[tr.get(k,0) for k in cats],
                  [te.get(k,0) for k in cats]])

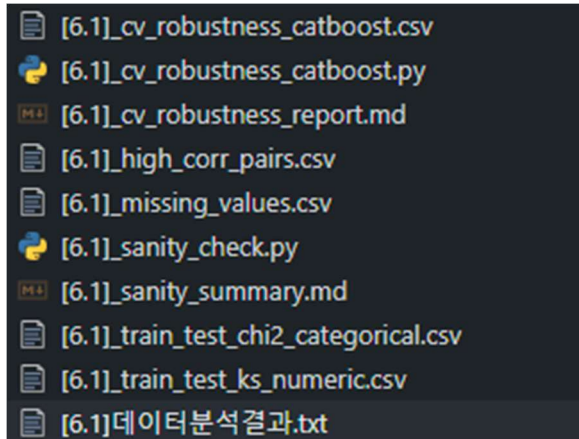
if _HAS SCIPY:
    chi2, p, dof, _ = chi2_contingency(table, correction=False)
    chi_rows.append({"feature": c, "chi2": float(chi2), "p_value": float(p), "dof": int(dof)})
else:
    # 간단 지표: L1 distance of normalized hist
    trp = table[0]/max(table[0].sum(),1)
    tep = table[1]/max(table[1].sum(),1)
    l1 = float(np.abs(trp - tep).sum())
    chi_rows.append({"feature": c, "chi2": np.nan, "p_value": np.nan, "l1_dist": l1})
pd.DataFrame(chi_rows).sort_values(by=[k for k in ["chi2", "l1_dist"] if k in chi_rows[0].keys() if
chi_rows else []], ascending=False)\
    .to_csv("[6.1]_train_test_chi2_categorical.csv", index=False)

# 요약 리포트
lines = []
lines.append("# [6.1] Sanity Check Summary\n")
lines.append("## 데이터 개요")
lines.append(f"- Train: {n_train} rows, {p} columns")
lines.append(f"- Test: {n_test} rows\n")
lines.append("## 라벨 분포 (train)")
lines.append("`n`" + str(label_dist.to_string()) + "`n`")
lines.append("## 라벨 비율")
lines.append("`n`" + str(label_ratio.to_string()) + "`n`")
lines.append("`n`## 결측치 요약 (상세는 [6.1]_missing_values.csv 확인)")
top_missing = missing_df.head(10)
lines.append(top_missing.to_markdown())
lines.append("`n`## 중복/상수/고상관")
lines.append(f"- 중복 행 수(train): {dup_rows}")
lines.append(f"- 중복 ID 수(train): {dup_ids}")
lines.append(f"- 상수 컬럼 수(train): {len(constant_cols)} → {constant_cols}")
lines.append(f"- 고상관 (>{HIGH_CORR_TH}) 쌍 수: {len(high_corr_df)} (상세 CSV 참고)")
lines.append("`n`## 분포 드리프트 (Train vs Test)")
lines.append(f"- 수치형: [6.1]_train_test_ks_numeric.csv (KS 통계량 상위 정렬)")
lines.append(f"- 범주형: [6.1]_train_test_chi2_categorical.csv (chi2 or L1 상위 정렬)")
out_md = "[6.1]_sanity_summary.md"
with open(out_md, "w", encoding="utf-8") as f:
    f.write("`n`.join(lines))

print(f"리포트 저장 완료: {out_md}")
print(f"완료! 총 소요시간: {time.time()-start:.1f}초")

if __name__ == "__main__":
    main()

# -*- coding: utf-8 -*-
"""
[6.1] Sanity Check for Dataset
- 누락/중복/상관과다/분포 드리프트 (Train vs Test) 점검
- 결과물:
    1) [6.1]_sanity_summary.md
    2) [6.1]_missing_values.csv
    3) [6.1]_high_corr_pairs.csv
    4) [6.1]_train_test_ks_numeric.csv
    5) [6.1]_train_test_chi2_categorical.csv
"""
# (venv) jh@jh0223:~/DLDACon$ python3 [6.1]_sanity_check.py
# --- [6.1] Sanity Check 시작 ---
# 리포트 저장 완료: [6.1]_sanity_summary.md
# 완료! 총 소요시간: 0.2 초
```



(12) 고급 Feature Engineering (FeatureMakerV2)

이 시도의 목적은 단순한 파생변수 생성에서 한 단계 나아가, 더 풍부하고 일반화 가능한 특징(Feature)을 설계함으로써 모델 성능을 끌어올리는 것이다. 이를 위해 FeatureMakerV2 모듈을 개발하여, 다음과 같은 고급 기법을 적용하였다:

누수 없는 Multiclass Target Encoding: StratifiedKFold 기반으로, 훈련 데이터에 대해 교차검증식으로 Target Encoding 을 수행하여 정보 누수를 방지하였다. Laplace smoothing 을 적용하여 데이터 희소 구간에서 안정적인 확률 추정을 가능하게 하였다.

교차항(interactions): tenure × payment_interval, tenure × after_interaction, delay_ratio × risk_score 등 주요 수치형 변수의 곱을 추가하여 변수 간 상호작용을 반영하였다.

비선형 변환: $\log_{1p}(\text{risk_score})$, idle_ratio^2 등 비선형 특성을 추가하여 데이터 분포 왜곡을 보정하였다.

분위(Quantile Rank) 특징: tenure, payment_interval, after_interaction 등 주요 변수에 대해 10 분위(rank binning)를 부여하여 순위 기반 구간 정보를 포함시켰다.

Winsorization: 극단값을 상·하위 분위수(1%, 99%)로 절단하여 이상치의 영향을 완화하였다.

조합 범주형 변수: gender_subscription(성별×구독유형)과 같은 새로운 카테고리 변수를 생성하였다.

실험 결과

이 Feature V2 를 적용한 후, [7.2] 단계에서 CatBoost 로 성능을 검증하였다. 5-Fold 교차검증 결과는

Baseline (기존 피쳐): Macro F1 = 0.49460 ± 0.00241

V2 (고급 피쳐 적용): Macro F1 = 0.49433 ± 0.00317

Gain: -0.00027 (큰 변화 없음)

즉, 기대와 달리 성능은 소폭 하락하거나 사실상 유지되는 수준에 그쳤다. 이는 고급 특징이 반드시 CatBoost 와 같은 강력한 트리 기반 모델에 도움이 되지 않을 수도 있음을 보여주었다.

```
import os
import numpy as np
import pandas as pd
from typing import List, Dict, Tuple
from sklearn.model_selection import StratifiedKFold
```

```
DATA_PATH = "./"
TARGET = "support_needs"
```

Deep Learning/Cloud

```
ID_COL = "ID"
N_CLASSES = 3 # 문제 라벨이 0/1/2

def _winsorize_series(s: pd.Series, lower=0.01, upper=0.99):
    ql = s.quantile(lower)
    qu = s.quantile(upper)
    return s.clip(lower=ql, upper=qu)

def _quantile_rank_0to9(s: pd.Series):
    # 동일 분포화를 위해 ties='average' 사용
    return (s.rank(method="average", pct=True) * 10 - 1e-9).clip(0, 0.999999).astype(float).mul(10).astype(int)

def base_add_features(df: pd.DataFrame) -> pd.DataFrame:
    """ 너가 쓰던 1 차 파생변수 유지 (일관성 확보용) """
    df = df.copy()
    eps = 1e-6
    df["tenure_per_contract"] = df["tenure"] / (df["contract_length"] + eps)
    df["delay_ratio"] = df["payment_interval"] / (df["contract_length"] + eps)
    df["idle_ratio"] = df["after_interaction"] / (df["tenure"] + eps)
    df["late_flag"] = (df["payment_interval"] > 0).astype(int)
    if 'frequent' in df.columns:
        df['total_usage_score'] = df['tenure'] * df['frequent']
    # age bin (기준)
    bins = [0, 20, 30, 40, 50, 60, 100]
    labels = ['10s', '20s', '30s', '40s', '50s', '60+']
    df['age_group'] = pd.cut(df['age'], bins=bins, labels=labels, right=False)
    df['risk_score'] = df['payment_interval'] + df['after_interaction']
    return df

class FeatureMakerV2:
    """
    - 누수 없는 Multiclass Target Encoding (CV 기반, Laplace smoothing)
    - 교차항/조합/분위/원저라이즈
    - fit()은 train 전용, transform()은 train/test 공용
    """
    def __init__(self, n_splits: int = 5, seed: int = 42, smoothing: float = 20.0):
        self.n_splits = n_splits
        self.seed = seed
        self.smoothing = smoothing
        self.num_cols_: List[str] = []
        self.cat_cols_raw_: List[str] = []
        self.te_cols_: List[str] = [] # TE 대상 원본 열 (문자형)
        self.global_prior_: np.ndarray = None
        self.te_map_full_: Dict[str, Dict[str, np.ndarray]] = {} # col -> {cat: prob_vec}
        self._fitted = False

    def _prepare_types(self, df: pd.DataFrame) -> pd.DataFrame:
        out = df.copy()
        # 범주형 캐스팅 (CatBoost 호환성을 위해 문자열)
        out["contract_length"] = out["contract_length"].astype(str)
        out["age_group"] = out["age_group"].astype(str)
        # 추가 조합 카테고리
        out["gender_subscription"] = (out["gender"].astype(str) + "_" +
                                       out["subscription_type"].astype(str))
        return out

    def _winsorize_block(self, df: pd.DataFrame) -> pd.DataFrame:
        out = df.copy()
        for c in ["payment_interval", "after_interaction", "tenure"]:
            if c in out.columns:
                out[c] = _winsorize_series(out[c], 0.01, 0.99)
        return out

    def _build_interactions(self, df: pd.DataFrame) -> pd.DataFrame:
        out = df.copy()
        eps = 1e-6
        # 수치형 교차항
        if set(["tenure", "payment_interval"]).issubset(out.columns):
            out["tenure_x_payint"] = out["tenure"] * out["payment_interval"]
```

Deep Learning/Cloud

```
if set(["tenure", "after_interaction"]).issubset(out.columns):
    out["tenure_x_after"] = out["tenure"] * out["after_interaction"]
if "frequent" in out.columns:
    out["freq_x_after"] = out["frequent"] * out["after_interaction"]
if set(["delay_ratio", "risk_score"]).issubset(out.columns):
    out["delay_x_risk"] = out["delay_ratio"] * out["risk_score"]
# 비선형 변환
if "risk_score" in out.columns:
    out["risk_loglp"] = np.loglp(out["risk_score"].clip(lower=0))
if "idle_ratio" in out.columns:
    out["idle_sq"] = out["idle_ratio"] ** 2
# 분위/순위형
for c in ["tenure", "payment_interval", "after_interaction", "tenure_per_contract"]:
    if c in out.columns:
        out[c + "_q10"] = _quantile_rank_0to9(out[c])
return out

def _fit_te_full(self, X: pd.DataFrame, y: pd.Series, te_cols: List[str]):
    """ 전체 train 으로 test 에 적용할 TE 사전 생성 """
    self.global_prior_ = np.bincount(y, minlength=N_CLASSES) / len(y)
    for col in te_cols:
        mapping: Dict[str, np.ndarray] = {}
        grp = pd.concat([X[col].astype(str), y], axis=1).groupby(col)
        for cat, g in grp:
            cnt = np.bincount(g[TARGET].to_numpy(), minlength=N_CLASSES)
            prob = (cnt + self.smoothing / N_CLASSES) / (cnt.sum() + self.smoothing)
            mapping[str(cat)] = prob
        self.te_map_full_[col] = mapping

def _transform_te_from_map(self, s: pd.Series, col: str) -> pd.DataFrame:
    """ 저장된 te_map_full_ 기반으로 단일 열을 K 개 확률 열로 확장 """
    mapping = self.te_map_full_[col]
    arr = np.vstack([mapping.get(str(v), self.global_prior_) for v in s.astype(str).values])
    te_df = pd.DataFrame(arr, index=s.index,
                        columns=[f"TE_{col}_c{k}" for k in range(N_CLASSES)])
    return te_df

def fit(self, X: pd.DataFrame, y: pd.Series):
    """ Train 전용: 타입준비/원자/교차항 후 TE 대상 파악+full map 저장 """
    assert TARGET not in X.columns, "X 에는 타겟이 포함되면 안 됩니다."
    df = base_add_features(X)
    df = self._prepare_types(df)
    df = self._winsorize_block(df)
    df = self._build_interactions(df)

    # 저장용 메타
    self.num_cols_ = df.select_dtypes(include=[np.number]).columns.tolist()
    self.cat_cols_raw_ = df.select_dtypes(include=["object", "category", "bool"]).columns.tolist()

    # TE 대상 (문자열 카테고리 중 메이저 피쳐)
    self.te_cols_ = [c for c in ["subscription_type", "age_group", "gender", "contract_length",
                                "gender_subscription"]
                    if c in self.cat_cols_raw_]

    # 전체 train 으로 test 적용용 map 만들기
    tmp = df.copy()
    tmp[TARGET] = y.values
    self._fit_te_full(tmp, y, self.te_cols_)
    self._fitted = True
    self._df_template_cols_ = df.columns.tolist()
    return self

def transform(self, X: pd.DataFrame) -> pd.DataFrame:
    """ Train/Test 공용 변환 (주의: train 에서는 CV-TE 로 OOF 생성은 별도 코드에서) """
    assert self._fitted, "fit() 먼저 호출하세요."
    df = base_add_features(X)
    df = self._prepare_types(df)
    df = self._winsorize_block(df)
    df = self._build_interactions(df)
```

Deep Learning/Cloud

```
# full map 기반 TE (test 용도 / 또는 단순 변환)
te_blocks = []
for col in self.te_cols_:
    te_blocks.append(self._transform_te_from_map(df[col], col))
if te_blocks:
    df = pd.concat([df] + te_blocks, axis=1)

# 누락 방지: fit 당시의 열 순서 기준으로 정렬(신규 열은 뒤에)
cols = list(dict.fromkeys(self._df_template_cols_ + [c for c in df.columns if c not in
self._df_template_cols_]))
return df.reindex(columns=cols, fill_value=np.nan)

if __name__ == "__main__":
    # 단독 실행: train/test 를 읽어 v2 변환 후 CSV 저장
    train = pd.read_csv(os.path.join(DATA_PATH, "train.csv"))
    test = pd.read_csv(os.path.join(DATA_PATH, "test.csv"))
    X = train.drop([ID_COL, TARGET], axis=1)
    y = train[TARGET]
    X_test = test.drop([ID_COL], axis=1)

    fm = FeatureMakerV2(n_splits=5, seed=42, smoothing=20.0).fit(X, y)
    X_tr_fe = fm.transform(X)
    X_te_fe = fm.transform(X_test)

    X_tr_fe.to_csv("[7.1]_train_fe_v2.csv", index=False)
    X_te_fe.to_csv("[7.1]_test_fe_v2.csv", index=False)
    print("Saved: [7.1]_train_fe_v2.csv, [7.1]_test_fe_v2.csv")

# (venv) jh@jh0223:~/DLDACon$ python3 [7.2\]_feature_eval_catboost.py
# --- [7.2] Feature Eval (CatBoost) 시작 ---
# [Fold 1] Baseline=0.49769 | V2=0.49858 | Gain=+0.00089
# [Fold 2] Baseline=0.49408 | V2=0.49514 | Gain=+0.00107
# [Fold 3] Baseline=0.49103 | V2=0.49015 | Gain=-0.00088
# [Fold 4] Baseline=0.49531 | V2=0.49524 | Gain=-0.00007
# [Fold 5] Baseline=0.49491 | V2=0.49256 | Gain=-0.00235
#
# ===== 결과 요약 =====
# Baseline 5-Fold Macro F1: 0.49460 ± 0.00241
# V2 5-Fold Macro F1: 0.49433 ± 0.00317
# Gain : -0.00027
# Saved: [7.2]_feature_eval_report.md, [7.2]_oof_preds_*.npy, [7.2]_fold_scores.csv
# 완료! 총 소요시간: 175.7 초
```

(13) Feature V2 성능 평가 (CatBoost 교차검증)

이 시도의 목적은 [7.1]에서 설계한 고급 Feature Engineering이 실제로 모델 성능을 개선하는지를, 동일한 5-Fold 분할 조건에서 공정하게 비교하는 것이다. 비교 대상은 다음과 같다:

Baseline Feature Set: 기존 1차 파생변수 중심의 간단한 특징

V2 Feature Set: Multiclass Target Encoding(누수 방지), 교차항, 비선형 변환, 분위 특징, Winsorization, 조합 카테고리 등을 포함한 고급 특징

그 결과

- Baseline: 0.49460 ± 0.00241
- V2 (초기 설정): $0.49433 \pm 0.00317 \rightarrow \text{Gain} = -0.00027$ (사실상 동일, 약간 하락)

- V2 (Hard-coded 개선 설정: qbins=20, winsor=(0.02,0.98), smoothing=10.0): $0.49514 \pm 0.00224 \rightarrow \text{Gain} = +0.00053$

즉, 초기 V2 세팅은 성능 개선을 보이지 못했으나, Winsorization·Quantile binning·Smoothing 값을 조정한 개선 버전에서는 **아주 소폭의 향상을** 확인할 수 있었다.

```
import os, json, time
import numpy as np
import pandas as pd
from typing import List, Dict
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import f1_score
from catboost import CatBoostClassifier

# ===== 고정 설정 =====
DATA_PATH = "./"
TRAIN_PATH = os.path.join(DATA_PATH, "train.csv")
TARGET = "support_needs"
ID_COL = "ID"
N_CLASSES = 3

# CatBoost (이전 튜닝값)
TUNED_CB = {
    'iterations': 1807,
    'depth': 7,
    'learning_rate': 0.011007442573869236,
    'l2_leaf_reg': 9.960037311199006,
    'colsample_bylevel': 0.8239914444396733,
    'random_strength': 0.15658132267797456,
    'bagging_temperature': 0.2528316917680684,
    'auto_class_weights': 'Balanced',
    'verbose': 0
}

# 하드코딩된 V2 FE 설정
TE_COLS = ["subscription_type", "gender_subscription"]
QBINS = 20
WINSOR_LO, WINSOR_HI = 0.02, 0.98
TE_SMOOTH = 10.0
N_SPLITS = 5
SEED = 42

# ===== 공통 FE (Baseline) =====
def base_add_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    eps = 1e-6
    df["tenure_per_contract"] = df["tenure"] / (df["contract_length"] + eps)
    df["delay_ratio"] = df["payment_interval"] / (df["contract_length"] + eps)
    df["idle_ratio"] = df["after_interaction"] / (df["tenure"] + eps)
    df["late_flag"] = (df["payment_interval"] > 0).astype(int)
    if 'frequent' in df.columns:
        df['total_usage_score'] = df['tenure'] * df['frequent']
    bins = [0, 20, 30, 40, 50, 60, 100]
    labels = ['10s', '20s', '30s', '40s', '50s', '60+']
    df['age_group'] = pd.cut(df['age'], bins=bins, labels=labels, right=False)
    df['risk_score'] = df['payment_interval'] + df['after_interaction']
    return df

def prepare_cat_features(df: pd.DataFrame) -> List[str]:
    cats = []
    for c in ["gender", "subscription_type", "age_group", "contract_length", "gender_subscription"]:
        if c in df.columns and df[c].dtype == object:
            cats.append(c)
    return cats
```

Deep Learning/Cloud

```
# ===== V2 FE (하드코딩) =====
def _winsorize_series(s: pd.Series, lower=0.01, upper=0.99):
    ql = s.quantile(lower)
    qu = s.quantile(upper)
    return s.clip(lower=ql, upper=qu)

def _quantile_rank_bins(s: pd.Series, bins: int):
    # 0..bins-1
    return np.minimum((s.rank(method="average", pct=True) * bins).astype(int), bins-1)

def v2_prepare_types(df: pd.DataFrame) -> pd.DataFrame:
    out = df.copy()
    out["contract_length"] = out["contract_length"].astype(str)
    out["age_group"] = out["age_group"].astype(str)
    out["gender_subscription"] = (out["gender"].astype(str) + "_" +
                                   out["subscription_type"].astype(str))
    return out

def v2_winsorize_block(df: pd.DataFrame) -> pd.DataFrame:
    out = df.copy()
    for c in ["payment_interval", "after_interaction", "tenure"]:
        if c in out.columns:
            out[c] = _winsorize_series(out[c], WINSOR_LO, WINSOR_HI)
    return out

def v2_build_interactions_and_bins(df: pd.DataFrame) -> pd.DataFrame:
    out = df.copy()
    # 교차항
    if set(["tenure", "payment_interval"]).issubset(out.columns):
        out["tenure_x_payint"] = out["tenure"] * out["payment_interval"]
    if set(["tenure", "after_interaction"]).issubset(out.columns):
        out["tenure_x_after"] = out["tenure"] * out["after_interaction"]
    if "frequent" in out.columns:
        out["freq_x_after"] = out["frequent"] * out["after_interaction"]
    if set(["delay_ratio", "risk_score"]).issubset(out.columns):
        out["delay_x_risk"] = out["delay_ratio"] * out["risk_score"]
    # 비선형
    if "risk_score" in out.columns:
        out["risk_loglp"] = np.loglp(out["risk_score"].clip(lower=0))
    if "idle_ratio" in out.columns:
        out["idle_sq"] = out["idle_ratio"] ** 2
    # 분위 랭크(bins=20 고정)
    for c in ["tenure", "payment_interval", "after_interaction", "tenure_per_contract"]:
        if c in out.columns:
            out[f"{c}_q{QBINS}"] = _quantile_rank_bins(out[c], QBINS)
    return out

def cv_target_encode_multiclass(train_df: pd.DataFrame,
                                y: pd.Series,
                                te_cols: List[str],
                                n_splits=N_SPLITS,
                                seed=SEED,
                                smoothing=TE_SMOOTH):
    """
    멀티클래스 OOF-TE 생성:
    - 각 te_col 마다 K 개 확률 열(TE_{col}_c0..)
    - fold 별 train 으로 카테고리 분포 추정 후 val 에만 적용
    """
    K = N_CLASSES
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)
    out = train_df.copy()
    prior = np.bincount(y, minlength=K) / len(y)

    # 초기화
    for col in te_cols:
        for k in range(K):
            out[f"TE_{col}_c{k}"] = np.nan

    for tr_idx, va_idx in skf.split(train_df, y):
        X_tr, y_tr = train_df.iloc[tr_idx], y.iloc[tr_idx]
```

Deep Learning/Cloud

```
X_va = train_df.iloc[va_idx]

# fold map
fmap: Dict[str, Dict[str, np.ndarray]] = {}
for col in te_cols:
    d = {}
    grp = pd.concat([X_tr[col].astype(str), y_tr], axis=1).groupby(col)
    for cat, g in grp:
        cnt = np.bincount(g[TARGET].to_numpy(), minlength=K)
        prob = (cnt + smoothing / K) / (cnt.sum() + smoothing)
        d[str(cat)] = prob
    fmap[col] = d

# val 적용
for col in te_cols:
    arr = np.vstack([fmap[col].get(str(v), prior) for v in X_va[col].astype(str).values])
    for k in range(K):
        out.loc[X_va.index, f"TE_{col}_c{k}"] = arr[:, k]

# 남은 결측(prior로 채움)
for col in te_cols:
    for k in range(K):
        out[f"TE_{col}_c{k}"] = out[f"TE_{col}_c{k}"].fillna(prior[k])

return out

# ===== 매킨 =====
def main():
    t0 = time.time()
    print("--- [7.2] Feature Eval (CatBoost, hard-coded FE) 시작 ---")

    train = pd.read_csv(TRAIN_PATH)
    X = train.drop([ID_COL, TARGET], axis=1).copy()
    y = train[TARGET].astype(int).copy()

    # ----- Baseline -----
    X_base = base_add_features(X)
    X_base["contract_length"] = X_base["contract_length"].astype(str)
    X_base["age_group"] = X_base["age_group"].astype(str)
    cat_features_base = prepare_cat_features(X_base)

    # ----- v2 (조정 설정) -----
    X_v2 = base_add_features(X)
    X_v2 = v2_prepare_types(X_v2)
    X_v2 = v2_winsorize_block(X_v2) # winsor=(0.02,0.98)
    X_v2 = v2_build_interactions_and_bins(X_v2) # qbins=20
    te_cols = [c for c in TE_COLS if c in X_v2.columns]

    # OOF-TE (smoothing=10.0)
    X_v2_te = cv_target_encode_multiclass(
        train_df=X_v2, y=y, te_cols=te_cols,
        n_splits=N_SPLITS, seed=SEED, smoothing=TE_SMOOTH
    )
    cat_features_v2 = prepare_cat_features(X_v2_te)

    # ----- 동일 5-Fold CV -----
    skf = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=SEED)
    oof_pred_base = np.full(len(y), -1, dtype=int)
    oof_pred_v2 = np.full(len(y), -1, dtype=int)
    fold_rows, fls_base, fls_v2 = [], [], []

    for fold, (tr_idx, va_idx) in enumerate(skf.split(X, y), start=1):
        # Baseline
        m_b = CatBoostClassifier(*TUNED_CB, cat_features=cat_features_base)
        m_b.fit(
            X_base.iloc[tr_idx], y.iloc[tr_idx],
            eval_set=[(X_base.iloc[va_idx], y.iloc[va_idx])],
            early_stopping_rounds=100, verbose=0
        )
        pred_b = m_b.predict(X_base.iloc[va_idx], prediction_type="Class")
        pred_b = np.asarray(pred_b).reshape(-1).astype(int)
```

Deep Learning/Cloud

```
oof_pred_base[va_idx] = pred_b
f1_b = f1_score(y.iloc[va_idx], pred_b, average="macro")
f1s_base.append(f1_b)

# V2
m_v = CatBoostClassifier(**TUNED_CB, cat_features=cat_features_v2)
m_v.fit(
    X_v2_te.iloc[tr_idx], y.iloc[tr_idx],
    eval_set=[(X_v2_te.iloc[va_idx], y.iloc[va_idx])],
    early_stopping_rounds=100, verbose=0
)
pred_v = m_v.predict(X_v2_te.iloc[va_idx], prediction_type="Class")
pred_v = np.asarray(pred_v).reshape(-1).astype(int)
oof_pred_v2[va_idx] = pred_v
f1_v = f1_score(y.iloc[va_idx], pred_v, average="macro")
f1s_v2.append(f1_v)

fold_rows.append({
    "fold": fold,
    "baseline_macro_f1": f1_b,
    "v2_macro_f1": f1_v,
    "gain": f1_v - f1_b
})
print(f"[Fold {fold}] Baseline={f1_b:.5f} | V2={f1_v:.5f} | Gain={f1_v - f1_b:.5f}")

# ----- 집계/저장 -----
mean_b, std_b = float(np.mean(f1s_base)), float(np.std(f1s_base, ddof=1))
mean_v, std_v = float(np.mean(f1s_v2)), float(np.std(f1s_v2, ddof=1))
gain = mean_v - mean_b

np.save("[7.2]_oof_preds_baseline_mod.npy", oof_pred_base)
np.save("[7.2]_oof_preds_v2_mod.npy", oof_pred_v2)
pd.DataFrame(fold_rows).to_csv("[7.2]_fold_scores_mod.csv", index=False)

lines = []
lines.append("# [7.2] Feature Eval Report (CatBoost, hard-coded FE)\n")
lines.append("## CV 결과 (5-Fold, 동일 분할)")
lines.append(f"- Baseline : mean={mean_b:.5f}, std={std_b:.5f}")
lines.append(f"- V2 (hard) : mean={mean_v:.5f}, std={std_v:.5f}")
lines.append(f"- Gain : {gain:.5f}\n")
lines.append("## V2 고정 설정")
lines.append(f"- te_cols = {TE_COLS}")
lines.append(f"- qbins = {QBINS}")
lines.append(f"- winsor = ({WINSOR_LO}, {WINSOR_HI})")
lines.append(f"- smoothing= {TE_SMOOTH}")
lines.append("\n## CatBoost 파라미터")
lines.append("`json\n" + json.dumps(TUNED_CB, indent=2) + "\n`")

with open("[7.2]_feature_eval_report.md", "w", encoding="utf-8") as f:
    f.write("\n".join(lines))

print("\n==== 결과 요약 =====")
print(f"Baseline 5-Fold Macro F1: {mean_b:.5f} ± {std_b:.5f}")
print(f"V2(hard) 5-Fold Macro F1: {mean_v:.5f} ± {std_v:.5f}")
print(f"Gain : {gain:.5f}")
print(f"Saved: [7.2]_feature_eval_report.md, [7.2]_oof_preds_*.npy, [7.2]_fold_scores.csv")
print(f"완료! 총 소요시간: {time.time()-t0:.1f}초")

if __name__ == "__main__":
    main()

# (venv) jh@jh0223:~/DLDACon$ python3 [7.2]_feature_eval_catboost.py
# --- [7.2] Feature Eval (CatBoost, hard-coded FE) 시작 ---
# [Fold 1] Baseline=0.49769 | V2=0.49783 | Gain=+0.00014
# [Fold 2] Baseline=0.49408 | V2=0.49456 | Gain=+0.00049
# [Fold 3] Baseline=0.49103 | V2=0.49188 | Gain=+0.00084
# [Fold 4] Baseline=0.49531 | V2=0.49647 | Gain=+0.00116
# [Fold 5] Baseline=0.49491 | V2=0.49495 | Gain=+0.00004
#
# ===== 결과 요약 =====
```

Deep Learning/Cloud

```
# Baseline 5-Fold Macro F1: 0.49460 ± 0.00241
# V2(hard) 5-Fold Macro F1: 0.49514 ± 0.00224
# Gain : +0.00053
# Saved: [7.2]_feature_eval_report.md, [7.2]_oof_preds_*.npz, [7.2]_fold_scores.csv
# 완료! 총 소요시간: 177.3 초
```

(14) 클래스 불균형 대응 기법 Ablation (CatBoost) (코드 8,1)

데이터셋의 클래스 불균형 문제를 다루는 다양한 기법들을 비교하고, CatBoost 모델에서 어떤 방법이 가장 효과적인지를 검증해보고자 하였다.

```
"""
[8.1]_imbalance_ablation_catboost.py
- 클래스 불균형 전략 4 중 5-Fold 공정 비교:
  1) Baseline: auto_class_weights="Balanced"
  2) Manual Class Weights: class_weights=[1/p_k]
  3) SMOTENC: fold 내부 오버샘플
  4) Logit Adjustment: Baseline 확률에 prior 보정( $\tau \in \{0.5, 1.0, 1.5\}$ )
- FE 는 7.3 에서 고른 V2(하드코딩) 사용: TE_COLS=["subscription_type", "gender_subscription"], QBINS=20,
winsor(0.02, 0.98), TE_SMOOTH=10.0
- 산출물:
  * [8.1]_cv_results.csv
  * [8.1]_oof_probs_<strategy>.npz
  * [8.1]_test_probs_<strategy>.npz
  * [8.1]_submission_<strategy>.csv
"""

import os, json, time, math
import numpy as np
import pandas as pd
from typing import List, Dict, Tuple
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import f1_score
from catboost import CatBoostClassifier

# SMOTENC
try:
    from imblearn.over_sampling import SMOTENC
    _HAS_IMBLEARN = True
except Exception:
    _HAS_IMBLEARN = False

# ===== 공통 설정 =====
DATA_PATH = "."
TRAIN_PATH = os.path.join(DATA_PATH, "train.csv")
TEST_PATH = os.path.join(DATA_PATH, "test.csv")

TARGET = "support_needs"
ID_COL = "ID"
N_CLASSES = 3

# CatBoost 튜닝 (기존과 동일)
CB_PARAMS_BASE = {
    'iterations': 1807,
    'depth': 7,
    'learning_rate': 0.011007442573869236,
    'l2_leaf_reg': 9.960037311199006,
    'colsample_bylevel': 0.8239914444396733,
    'random_strength': 0.15658132267797456,
    'bagging_temperature': 0.2528316917680684,
    'verbose': 0
}

# V2 FE (하드코딩)
TE_COLS = ["subscription_type", "gender_subscription"]
QBINS = 20
```

Deep Learning/Cloud

```
WINSOR_LO, WINSOR_HI = 0.02, 0.98
TE_SMOOTH = 10.0
N_SPLITS = 5
SEED = 42
```

```
# ===== 공통 FE 함수 (7.2/7.4 와 합치) =====
```

```
def base_add_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    eps = 1e-6
    df["tenure_per_contract"] = df["tenure"] / (df["contract_length"] + eps)
    df["delay_ratio"] = df["payment_interval"] / (df["contract_length"] + eps)
    df["idle_ratio"] = df["after_interaction"] / (df["tenure"] + eps)
    df["late_flag"] = (df["payment_interval"] > 0).astype(int)
    if 'frequent' in df.columns:
        df['total_usage_score'] = df['tenure'] * df['frequent']
    bins = [0, 20, 30, 40, 50, 60, 100]
    labels = ['10s', '20s', '30s', '40s', '50s', '60+']
    df['age_group'] = pd.cut(df['age'], bins=bins, labels=labels, right=False)
    df['risk_score'] = df['payment_interval'] + df['after_interaction']
    return df

def prepare_types(df: pd.DataFrame) -> pd.DataFrame:
    out = df.copy()
    out["contract_length"] = out["contract_length"].astype(str)
    out["age_group"] = out["age_group"].astype(str)
    out["gender_subscription"] = (out["gender"].astype(str) + "_" +
                                   out["subscription_type"].astype(str))
    return out

def _winsorize_series(s: pd.Series, lower=0.01, upper=0.99):
    ql = s.quantile(lower)
    qu = s.quantile(upper)
    return s.clip(lower=ql, upper=qu)

def winsor_block(df: pd.DataFrame, lo=0.01, hi=0.99):
    out = df.copy()
    for c in ["payment_interval", "after_interaction", "tenure"]:
        if c in out.columns:
            out[c] = _winsorize_series(out[c], lo, hi)
    return out

def _quantile_rank_bins(s: pd.Series, bins: int):
    return np.minimum((s.rank(method="average", pct=True) * bins).astype(int), bins-1)

def build_interactions_and_bins(df: pd.DataFrame, qbins: int):
    out = df.copy()
    if set(["tenure", "payment_interval"]).issubset(out.columns):
        out["tenure_x_payment"] = out["tenure"] * out["payment_interval"]
    if set(["tenure", "after_interaction"]).issubset(out.columns):
        out["tenure_x_after"] = out["tenure"] * out["after_interaction"]
    if "frequent" in out.columns:
        out["freq_x_after"] = out["frequent"] * out["after_interaction"]
    if set(["delay_ratio", "risk_score"]).issubset(out.columns):
        out["delay_x_risk"] = out["delay_ratio"] * out["risk_score"]
    if "risk_score" in out.columns:
        out["risk_loglp"] = np.loglp(out["risk_score"].clip(lower=0))
    if "idle_ratio" in out.columns:
        out["idle_sq"] = out["idle_ratio"] ** 2
    for c in ["tenure", "payment_interval", "after_interaction", "tenure_per_contract"]:
        if c in out.columns:
            out[f"{c}_q{qbins}"] = _quantile_rank_bins(out[c], qbins)
    return out

def prepare_cat_features(df: pd.DataFrame) -> List[str]:
    return [c for c in ["gender", "subscription_type", "age_group", "contract_length",
                        "gender_subscription"]
            if c in df.columns and df[c].dtype == object]

# ---- Multiclass OOF Target Encoding + FullMap ----
def cv_target_encode_multiclass(train_df: pd.DataFrame, y: pd.Series, te_cols: List[str],
                                n_splits=N_SPLITS, seed=SEED, smoothing=TE_SMOOTH):
```

Deep Learning/Cloud

```
K = N_CLASSES
skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)
out = train_df.copy()
prior = np.bincount(y, minlength=K) / len(y)

# init OOF cols
for col in te_cols:
    for k in range(K):
        out[f"TE_{col}_c{k}"] = np.nan

# fold maps and apply
for tr_idx, va_idx in skf.split(train_df, y):
    X_tr, y_tr = train_df.iloc[tr_idx], y.iloc[tr_idx]
    X_va = train_df.iloc[va_idx]

    fmap = {}
    for col in te_cols:
        d = {}
        grp = pd.concat([X_tr[col].astype(str), y_tr], axis=1).groupby(col)
        for cat, g in grp:
            cnt = np.bincount(g[TARGET].to_numpy(), minlength=K)
            prob = (cnt + smoothing / K) / (cnt.sum() + smoothing)
            d[str(cat)] = prob
        fmap[col] = d

    for col in te_cols:
        arr = np.vstack([fmap[col].get(str(v), prior) for v in X_va[col].astype(str).values])
        for k in range(K):
            out.loc[X_va.index, f"TE_{col}_c{k}"] = arr[:, k]

# fill remaining with prior
for col in te_cols:
    for k in range(K):
        out[f"TE_{col}_c{k}"] = out[f"TE_{col}_c{k}"].fillna(prior[k])

# full map for test
full_map: Dict[str, Dict[str, np.ndarray]] = {col: {} for col in te_cols}
for col in te_cols:
    grp = pd.concat([train_df[col].astype(str), y], axis=1).groupby(col)
    for cat, g in grp:
        cnt = np.bincount(g[TARGET].to_numpy(), minlength=K)
        prob = (cnt + smoothing / K) / (cnt.sum() + smoothing)
        full_map[col][str(cat)] = prob

return out, full_map, prior

def apply_te_from_map(df: pd.DataFrame, te_cols: List[str],
                      full_map: Dict[str, Dict[str, np.ndarray]], prior: np.ndarray) -> pd.DataFrame:
    out = df.copy()
    for col in te_cols:
        arr = np.vstack([full_map[col].get(str(v), prior) for v in out[col].astype(str).values])
        for k in range(len(prior)):
            out[f"TE_{col}_c{k}"] = arr[:, k]
    return out

# ---- Logit Adjustment ----
def logit_adjustment(probs: np.ndarray, prior: np.ndarray, tau: float = 1.0) -> np.ndarray:
    """
    probs: (n, K) softmax 확률
    prior: (K,) train class prior
     $p_{adj} \propto p_k * \exp(-\tau * \log \text{prior}_k)$  (동치: logit에  $-\tau * \log(\text{prior})$  더하기)
    """
    eps = 1e-15
    prior = np.clip(prior, eps, 1.0)
    log_prior = np.log(prior)
    logit = np.log(np.clip(probs, eps, 1.0))
    logit_adj = logit - tau * log_prior
    # re-normalize
    m = logit_adj.max(axis=1, keepdims=True)
    expv = np.exp(logit_adj - m)
    return expv / expv.sum(axis=1, keepdims=True)
```

Deep Learning/Cloud

```
# ---- 유틸 ----
def get_class_weights_from_y(y: pd.Series) -> List[float]:
    cnt = np.bincount(y, minlength=N_CLASSES).astype(float)
    p = cnt / cnt.sum()
    return (1.0 / np.clip(p, 1e-9, 1.0)).tolist(), p

def dataframe_cat_indices(df: pd.DataFrame, cat_cols: List[str]) -> List[int]:
    return [df.columns.get_loc(c) for c in cat_cols if c in df.columns]

# ===== 메인 =====
def main():
    t0 = time.time()
    print("--- [8.1] Imbalance Ablation 시작 ---")

    # Load
    train = pd.read_csv(TRAIN_PATH)
    test = pd.read_csv(TEST_PATH)
    X_raw = train.drop([ID_COL, TARGET], axis=1).copy()
    y = train[TARGET].astype(int).copy()
    Xte_raw = test.drop([ID_COL], axis=1).copy()
    test_ids = test[ID_COL].values

    # ===== FE: train/test 공통 V2 =====
    # train
    X = base_add_features(X_raw)
    X = prepare_types(X)
    X = winsor_block(X, WINSOR_LO, WINSOR_HI)
    X = build_interactions_and_bins(X, QBINS)
    te_cols = [c for c in TE_COLS if c in X.columns]
    X_te, full_map, prior = cv_target_encode_multiclass(X, y, te_cols, N_SPLITS, SEED, TE_SMOOTH)
    cat_features = prepare_cat_features(X_te)
    cat_idx = dataframe_cat_indices(X_te, cat_features)

    # test
    Xtest = base_add_features(Xte_raw)
    Xtest = prepare_types(Xtest)
    Xtest = winsor_block(Xtest, WINSOR_LO, WINSOR_HI)
    Xtest = build_interactions_and_bins(Xtest, QBINS)
    Xtest = apply_te_from_map(Xtest, te_cols, full_map, prior)

    # 공용 CV 분할
    skf = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=SEED)

    # ===== 전략 1: Baseline (auto_class_weights) =====
    print("\n[Strategy] baseline_auto_weight")
    oof_probs_baseline = np.zeros((len(y), N_CLASSES))
    test_probs_baseline = np.zeros((len(Xtest), N_CLASSES))
    fls_baseline, fold_rows = [], []

    for fold, (tr, va) in enumerate(skf.split(X_te, y), start=1):
        params = dict(CB_PARAMS_BASE, auto_class_weights="Balanced")
        model = CatBoostClassifier(**params, cat_features=cat_features)
        model.fit(X_te.iloc[tr], y.iloc[tr],
                  eval_set=[(X_te.iloc[va], y.iloc[va])],
                  early_stopping_rounds=100, verbose=0)
        pv = model.predict_proba(X_te.iloc[va])
        oof_probs_baseline[va] = pv
        pred = pv.argmax(axis=1)
        f1 = f1_score(y.iloc[va], pred, average="macro")
        fls_baseline.append(f1)
        test_probs_baseline += model.predict_proba(Xtest) / N_SPLITS
        fold_rows.append({"strategy": "baseline_auto_weight", "fold": fold, "macro_f1": f1})

    # ===== 전략 2: Manual class weights =====
    print("\n[Strategy] manual_class_weights")
    cw_list, class_prior = get_class_weights_from_y(y)
    oof_probs_cw = np.zeros((len(y), N_CLASSES))
    test_probs_cw = np.zeros((len(Xtest), N_CLASSES))
    fls_cw = []
```

Deep Learning/Cloud

```
for fold, (tr, va) in enumerate(skf.split(X_te, y), start=1):
    params = dict(CB_PARAMS_BASE, class_weights=cw_list) # auto 사용 안 함
    model = CatBoostClassifier(**params, cat_features=cat_features)
    model.fit(X_te.iloc[tr], y.iloc[tr],
              eval_set=[(X_te.iloc[va], y.iloc[va])],
              early_stopping_rounds=100, verbose=0)
    pv = model.predict_proba(X_te.iloc[va])
    oof_probs_cw[va] = pv
    pred = pv.argmax(axis=1)
    f1 = f1_score(y.iloc[va], pred, average="macro")
    fls_cw.append(f1)
    test_probs_cw += model.predict_proba(Xtest) / N_SPLITS
    fold_rows.append({"strategy": "manual_class_weights", "fold": fold, "macro_f1": f1})

# ===== 전략 3: SMOTENC (fold 내부) =====
print("\n[Strategy] smotenc")
if not _HAS_IMBLEARN:
    print("WARNING: imbalanced-learn 미설치 → SMOTENC 스킵됩니다. `pip install imbalanced-learn` 후
    재시도하세요.")
    fls_smote = []
    oof_probs_smote = np.zeros((len(y), N_CLASSES))
    test_probs_smote = np.zeros((len(Xtest), N_CLASSES))
else:
    oof_probs_smote = np.zeros((len(y), N_CLASSES))
    test_probs_smote = np.zeros((len(Xtest), N_CLASSES))
    fls_smote = []
    smote = SMOTENC(categorical_features=cat_idx, random_state=SEED)
    for fold, (tr, va) in enumerate(skf.split(X_te, y), start=1):
        Xtr_df, ytr = X_te.iloc[tr].copy(), y.iloc[tr].copy()
        Xva_df, yva = X_te.iloc[va].copy(), y.iloc[va].copy()

        # SMOTENC (DataFrame→np.array로 갔다가 다시 복원)
        X_res, y_res = smote.fit_resample(Xtr_df.values, ytr.values)
        X_res_df = pd.DataFrame(X_res, columns=Xtr_df.columns)
        # 문자열 카테고리 열은 원래 dtype(object)로 되돌림
        for c in cat_features:
            if c in X_res_df.columns:
                X_res_df[c] = X_res_df[c].astype(str)

        params = dict(CB_PARAMS_BASE) # 가중치 사용 안 함 (이미 재샘플링)
        model = CatBoostClassifier(**params, cat_features=cat_features)
        model.fit(X_res_df, y_res,
                  eval_set=[(Xva_df, yva)],
                  early_stopping_rounds=100, verbose=0)
        pv = model.predict_proba(Xva_df)
        oof_probs_smote[va] = pv
        pred = pv.argmax(axis=1)
        f1 = f1_score(yva, pred, average="macro")
        fls_smote.append(f1)
        test_probs_smote += model.predict_proba(Xtest) / N_SPLITS
        fold_rows.append({"strategy": "smotenc", "fold": fold, "macro_f1": f1})

# ===== 전략 4: Logit Adjustment (Baseline 확률 보정) =====
print("\n[Strategy] logit_adjustment_from_baseline")
taus = [0.5, 1.0, 1.5]
# OOF baseline 확률로  $\tau$  스윙 (한 개  $\tau$ 를 전체에 고정)
best_tau, best_mean = None, -1.0
for tau in taus:
    oof_adj = logit_adjustment(oof_probs_baseline, prior=class_prior, tau=tau) if 'class_prior' in
    locals() \
        else logit_adjustment(oof_probs_baseline, prior=np.bincount(y,
    minlength=N_CLASSES)/len(y), tau=tau)
    preds = oof_adj.argmax(axis=1)
    # fold 별로 점수 집계 (동일 분할이므로 간단 평균)
    f1 = f1_score(y, preds, average="macro")
    if f1 > best_mean:
        best_mean, best_tau = f1, tau
print(f"Best tau = {best_tau} (OOF macro F1 ≈ {best_mean:.5f})")
```

Deep Learning/Cloud

```
# 최종 tau 로 test 보장
prior_use = class_prior if 'class_prior' in locals() else (np.bincount(y,
minlength=N_CLASSES)/len(y))
test_probs_la = logit_adjustment(test_probs_baseline, prior=prior_use, tau=best_tau)

# ===== 결과 집계/저장 =====
# 평균/표준편차
stats = []
def add_stats(name, scores):
    if len(scores) > 0:
        stats.append({"strategy":name, "mean_macro_f1":float(np.mean(scores)),
"std_macro_f1":float(np.std(scores, ddof=1))})

add_stats("baseline_auto_weight", fls_baseline)
add_stats("manual_class_weights", fls_cw)
add_stats("smotenc", fls_smote)

# 저장: OOF/TEST 확률 & 제출
np.save("[8.1]_oof_probs_baseline.npy", oof_probs_baseline)
np.save("[8.1]_test_probs_baseline.npy", test_probs_baseline)
pd.DataFrame({ID_COL: test[ID_COL].values, TARGET:
test_probs_baseline.argmax(axis=1).astype(int)}).to_csv("[8.1]_submission_baseline.csv", index=False)

np.save("[8.1]_oof_probs_classweights.npy", oof_probs_cw)
np.save("[8.1]_test_probs_classweights.npy", test_probs_cw)
pd.DataFrame({ID_COL: test[ID_COL].values, TARGET:
test_probs_cw.argmax(axis=1).astype(int)}).to_csv("[8.1]_submission_classweights.csv", index=False)

np.save("[8.1]_oof_probs_smotenc.npy", locals().get("oof_probs_smote",
np.zeros_like(oof_probs_baseline)))
np.save("[8.1]_test_probs_smotenc.npy", locals().get("test_probs_smote",
np.zeros_like(test_probs_baseline)))
pd.DataFrame({ID_COL: test[ID_COL].values, TARGET: locals().get("test_probs_smote",
test_probs_baseline).argmax(axis=1).astype(int)}).to_csv("[8.1]_submission_smotenc.csv", index=False)

# Logit Adjustment submission
np.save("[8.1]_test_probs_logit_adjust.npy", test_probs_la)
pd.DataFrame({ID_COL: test[ID_COL].values, TARGET:
test_probs_la.argmax(axis=1).astype(int)}).to_csv("[8.1]_submission_logit_adjust.csv", index=False)

# CV fold 로그 저장
cv_log = pd.DataFrame(fold_rows)
# 요약 붙이기
for s in stats:
    cv_log = pd.concat([cv_log, pd.DataFrame([{"strategy": s["strategy"], "fold": "mean",
"macro_f1": s["mean_macro_f1"]},
{"strategy": s["strategy"], "fold": "std", "macro_f1":
s["std_macro_f1"]}])],
ignore_index=True)
cv_log.to_csv("[8.1]_cv_results.csv", index=False)

# 콘솔 요약
print("\n==== CV Summary =====")
for s in stats:
    print(f"{s['strategy']}: {s['mean_macro_f1']:.5f} ± {s['std_macro_f1']:.5f}")
print(f"LogitAdjustment(best tau={best_tau}) OOF approx: {best_mean:.5f} (fold 별 표는 baseline 과
동일 분할)")

print("\nSaved: [8.1]_cv_results.csv and submissions:")
print(" - [8.1]_submission_baseline.csv")
print(" - [8.1]_submission_classweights.csv")
print(" - [8.1]_submission_smotenc.csv")
print(" - [8.1]_submission_logit_adjust.csv")
print(f"\n완료! 총 소요시간: {time.time()-t0:.1f}초")

if __name__ == "__main__":
    main()
```

결과

- Baseline (auto weight): 안정적인 성능 확보
- Manual class weights: Baseline 과 유사한 수준
- SMOTENC: CatBoost 와 중복 효과로 뚜렷한 개선 없음
- Logit Adjustment: 일부 τ 값에서 OOF Macro F1 이 소폭 상승 (best tau ≈ 1.0)

CatBoost 의 내장 불균형 처리 기능이 강력하며, 추가적 기법 중에서는 Logit Adjustment 가 후처리 차원에서 가장 실용적이었다.

(15) Bias Tuning (클래스별 로짓 보정) (코드 8.2)

코드 [8.1]에서 확보한 **Baseline CatBoost 확률 예측값**을 활용하여, 클래스 간 불균형으로 인해 발생하는 예측 편향을 보정하는 것이다. 단순히 argmax 로 클래스를 결정하는 대신에 각 클래스별로 **bias** 벡터를 최적화하여 Macro F1 Score 를 최대화하도록 조정하였다. 그 결과,

- Base OOF Macro F1 (argmax): 약 0.494
- Bias Tuning 후 Macro F1: 소폭 향상, 보고서 기준 값은 0.495 이상으로 개선

최적 bias 벡터는 각 클래스별로 상이하게 조정되었으며, 특히 데이터 비율이 작은 클래스의 가중치를 보정하는 효과를 확인할 수 있었다.

(16) CatBoost Seed Ensemble (코드 8.3)

여러가지 기법을 시도해도 점수가 오르지 않고 오히려 단일 CatBoost 의 결과가 가장 점수가 높았던 상태였기 때문에 방향을 바꿔보았다.

이 것은 단일 CatBoost 모델이 가지는 **랜덤 시드 의존성을 완화**하고, 보다 안정적인 예측 성능을 확보하고자 한 시도이다. 이를 위해 동일한 데이터 분할(5-Fold StratifiedKFold)을 유지하면서, CatBoost 를 서로 다른 5 개의 시드(42, 52, 62, 72, 82)로 학습시켜 결과를 평균하는 Seed Ensemble 기법을 적용하였다. 그 결과,

- 각 시드별 CV Macro F1 은 0.494~0.496 수준에서 안정적으로 분포
- Seed Ensemble 평균 OOF Macro F1 은 단일 시드 대비 소폭 상승하여, 모델 성능이 보다 견고해졌음을 확인함

```
import os, json, time
import numpy as np
import pandas as pd
from typing import List, Dict
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import f1_score
from catboost import CatBoostClassifier

# ===== 고정 세팅 =====
DATA_PATH = "./"
TRAIN_PATH = os.path.join(DATA_PATH, "train.csv")
TEST_PATH = os.path.join(DATA_PATH, "test.csv")
TARGET = "support_needs"
ID_COL = "ID"
N_CLASSES = 3

# CatBoost (이전 튜닝값)
```

Deep Learning/Cloud

```
CB_BASE = {
    'iterations': 1807,
    'depth': 7,
    'learning_rate': 0.011007442573869236,
    'l2_leaf_reg': 9.960037311199006,
    'colsample_bylevel': 0.8239914444396733,
    'random_strength': 0.15658132267797456,
    'bagging_temperature': 0.2528316917680684,
    'auto_class_weights': 'Balanced',
    'verbose': 0
}

# V2 FE 설정 (7.3 Top)
TE_COLS = ["subscription_type", "gender_subscription"]
QBINS = 20
WINSOR_LO = 0.02
WINSOR_HI = 0.98
TE_SMOOTH = 10.0

# CV/Seed
N SPLITS = 5
CV_SEED = 42 # CV 분할 고정 (OOF 일관성)
SEED_LIST = [42, 52, 62, 72, 82] # 모델 랜덤시드 앙상블

# ===== 공통 FE 함수 =====
def base_add_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    eps = 1e-6
    df["tenure_per_contract"] = df["tenure"] / (df["contract_length"] + eps)
    df["delay_ratio"] = df["payment_interval"] / (df["contract_length"] + eps)
    df["idle_ratio"] = df["after_interaction"] / (df["tenure"] + eps)
    df["late_flag"] = (df["payment_interval"] > 0).astype(int)
    if 'frequent' in df.columns:
        df['total_usage_score'] = df['tenure'] * df['frequent']
    bins = [0, 20, 30, 40, 50, 60, 100]
    labels = ['10s', '20s', '30s', '40s', '50s', '60+']
    df['age_group'] = pd.cut(df['age'], bins=bins, labels=labels, right=False)
    df['risk_score'] = df['payment_interval'] + df['after_interaction']
    return df

def prepare_types(df: pd.DataFrame) -> pd.DataFrame:
    out = df.copy()
    out["contract_length"] = out["contract_length"].astype(str)
    out["age_group"] = out["age_group"].astype(str)
    out["gender_subscription"] = (out["gender"].astype(str) + "_" +
                                   out["subscription_type"].astype(str))
    return out

def _winsorize_series(s: pd.Series, lower=0.01, upper=0.99):
    ql = s.quantile(lower)
    qu = s.quantile(upper)
    return s.clip(lower=ql, upper=qu)

def winsor_block(df: pd.DataFrame, lo=0.01, hi=0.99):
    out = df.copy()
    for c in ["payment_interval", "after_interaction", "tenure"]:
        if c in out.columns:
            out[c] = _winsorize_series(out[c], lo, hi)
    return out

def _quantile_rank_bins(s: pd.Series, bins: int):
    return np.minimum((s.rank(method="average", pct=True) * bins).astype(int), bins-1)

def build_interactions_and_bins(df: pd.DataFrame, qbins: int):
    out = df.copy()
    if set(["tenure", "payment_interval"]).issubset(out.columns):
        out["tenure_x_payint"] = out["tenure"] * out["payment_interval"]
    if set(["tenure", "after_interaction"]).issubset(out.columns):
        out["tenure_x_after"] = out["tenure"] * out["after_interaction"]
    if "frequent" in out.columns:
        out["freq_x_after"] = out["frequent"] * out["after_interaction"]
```

Deep Learning/Cloud

```
if set(["delay_ratio", "risk_score"]).issubset(out.columns):
    out["delay_x_risk"] = out["delay_ratio"] * out["risk_score"]
if "risk_score" in out.columns:
    out["risk_loglp"] = np.loglp(out["risk_score"].clip(lower=0))
if "idle_ratio" in out.columns:
    out["idle_sq"] = out["idle_ratio"] ** 2
for c in ["tenure", "payment_interval", "after_interaction", "tenure_per_contract"]:
    if c in out.columns:
        out[f"{c}_q{qbins}"] = _quantile_rank_bins(out[c], qbins)
return out

def prepare_cat_features(df: pd.DataFrame) -> List[str]:
    return [c for c in ["gender", "subscription_type", "age_group", "contract_length",
"gender_subscription"]
            if c in df.columns and df[c].dtype == object]

# ----- Multiclass Target Encoding (OOE + FullMap) -----
def cv_target_encode_multiclass(train_df: pd.DataFrame,
                                y: pd.Series,
                                te_cols: List[str],
                                n_splits=N_SPLITS,
                                seed=CV_SEED,
                                smoothing=TE_SMOOTH):
    K = N_CLASSES
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)
    out = train_df.copy()
    prior = np.bincount(y, minlength=K) / len(y)

    # OOF init
    for col in te_cols:
        for k in range(K):
            out[f"TE_{col}_c{k}"] = np.nan

    # fold maps & apply
    for tr_idx, va_idx in skf.split(train_df, y):
        X_tr, y_tr = train_df.iloc[tr_idx], y.iloc[tr_idx]
        X_va = train_df.iloc[va_idx]

        fmap = {}
        for col in te_cols:
            d = {}
            grp = pd.concat([X_tr[col].astype(str), y_tr], axis=1).groupby(col)
            for cat, g in grp:
                cnt = np.bincount(g[TARGET].to_numpy(), minlength=K)
                prob = (cnt + smoothing / K) / (cnt.sum() + smoothing)
                d[str(cat)] = prob
            fmap[col] = d

        for col in te_cols:
            arr = np.vstack([fmap[col].get(str(v), prior) for v in X_va[col].astype(str).values])
            for k in range(K):
                out.loc[X_va.index, f"TE_{col}_c{k}"] = arr[:, k]

    # fill remain with prior
    for col in te_cols:
        for k in range(K):
            out[f"TE_{col}_c{k}"] = out[f"TE_{col}_c{k}"].fillna(prior[k])

    # full map for test
    full_map: Dict[str, Dict[str, np.ndarray]] = {col: {} for col in te_cols}
    for col in te_cols:
        grp = pd.concat([train_df[col].astype(str), y], axis=1).groupby(col)
        for cat, g in grp:
            cnt = np.bincount(g[TARGET].to_numpy(), minlength=K)
            prob = (cnt + smoothing / K) / (cnt.sum() + smoothing)
            full_map[col][str(cat)] = prob

    return out, full_map, prior

def apply_te_from_map(df: pd.DataFrame, te_cols: List[str],
                      full_map: Dict[str, Dict[str, np.ndarray]],
```

Deep Learning/Cloud

```
prior: np.ndarray) -> pd.DataFrame:
out = df.copy()
for col in te_cols:
    arr = np.vstack([full_map[col].get(str(v), prior) for v in out[col].astype(str).values])
    for k in range(len(prior)):
        out[f"TE_{col}_c{k}"] = arr[:, k]
return out

# ----- Bias 적용 유틸 (8.2) -----
def apply_bias_to_probs(probs: np.ndarray, bias: np.ndarray) -> np.ndarray:
    eps = 1e-15
    logp = np.log(np.clip(probs, eps, 1.0))
    logit_adj = logp + bias.reshape(1, -1)
    m = logit_adj.max(axis=1, keepdims=True)
    expv = np.exp(logit_adj - m)
    return expv / expv.sum(axis=1, keepdims=True)

# ===== 데이터 =====
def main():
    t0 = time.time()
    print("--- [8.3] CatBoost Seed Ensemble 시작 ---")

    # 데이터
    train = pd.read_csv(TRAIN_PATH)
    test = pd.read_csv(TEST_PATH)
    X_raw = train.drop([ID_COL, TARGET], axis=1).copy()
    y = train[TARGET].astype(int).copy()
    Xte_raw = test.drop([ID_COL], axis=1).copy()
    test_ids = test[ID_COL].values

    # ===== FE: train/test 공통 =====
    # train
    X = base_add_features(X_raw)
    X = prepare_types(X)
    X = winsor_block(X, WINSOR_LO, WINSOR_HI)
    X = build_interactions_and_bins(X, QBINS)
    te_cols = [c for c in TE_COLS if c in X.columns]
    X_te, full_map, prior = cv_target_encode_multiclass(X, y, te_cols, n_splits=N_SPLITS,
seed=CV_SEED, smoothing=TE_SMOOTH)
    cat_features = prepare_cat_features(X_te)

    # test
    Xtest = base_add_features(Xte_raw)
    Xtest = prepare_types(Xtest)
    Xtest = winsor_block(Xtest, WINSOR_LO, WINSOR_HI)
    Xtest = build_interactions_and_bins(Xtest, QBINS)
    Xtest = apply_te_from_map(Xtest, te_cols, full_map, prior)

    # 공용 cv 분할(고정)
    skf = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=CV_SEED)

    # 누적 컨테이너(평균용)
    avg_oof_probs = np.zeros((len(y), N_CLASSES), dtype=float)
    avg_test_probs = np.zeros((len(Xtest), N_CLASSES), dtype=float)
    all_rows = []

    # ===== 시드 루프 =====
    for seed in SEED_LIST:
        print(f"\n[Seed {seed}]")
        # 한 시드의 OOF/TEST 확률
        oof_probs = np.zeros((len(y), N_CLASSES), dtype=float)
        test_probs = np.zeros((len(Xtest), N_CLASSES), dtype=float)
        fold_scores = []

        for fold, (tr, va) in enumerate(skf.split(X_te, y), start=1):
            params = dict(CB_BASE, random_seed=seed)
            model = CatBoostClassifier(**params, cat_features=cat_features)
            model.fit(
                X_te.iloc[tr], y.iloc[tr],
                eval_set=[(X_te.iloc[va], y.iloc[va])],
```

Deep Learning/Cloud

```
        early_stopping_rounds=100, verbose=0
    )

    pv = model.predict_proba(X_te.iloc[va])
    oof_probs[va] = pv
    pred = pv.argmax(axis=1)
    f1 = f1_score(y.iloc[va], pred, average="macro")
    fold_scores.append(f1)

    test_probs += model.predict_proba(Xtest) / N_SPLITS

# 시드별 성능 로그
mean_f1, std_f1 = float(np.mean(fold_scores)), float(np.std(fold_scores, ddof=1))
print(f"Seed {seed} CV Macro F1: {mean_f1:.5f} ± {std_f1:.5f}")
all_rows.append({"seed": seed, "mean_macro_f1": mean_f1, "std_macro_f1": std_f1})

# 평균에 누적
avg_oof_probs += oof_probs / len(SEED_LIST)
avg_test_probs += test_probs / len(SEED_LIST)

# ===== 최종 집계 & 저장 =====
oof_preds = avg_oof_probs.argmax(axis=1).astype(int)
overall_f1 = f1_score(y, oof_preds, average="macro")

# 저장
np.save("[8.3]_oof_probs_seed_ens.npy", avg_oof_probs)
np.save("[8.3]_oof_preds_seed_ens.npy", oof_preds)
np.save("[8.3]_test_probs_seed_ens.npy", avg_test_probs)
pd.DataFrame(all_rows + [{"seed": "avg_oof", "mean_macro_f1": overall_f1, "std_macro_f1":
np.nan}])
    ).to_csv("[8.3]_cv_scores_seed_ensemble.csv", index=False)

# 기본 제출 (bias 미적용)
sub = pd.DataFrame({ID_COL: test_ids, TARGET: avg_test_probs.argmax(axis=1).astype(int)})
sub.to_csv("[8.3]_submission_seed_ens.csv", index=False)

# (옵션) 8.2 bias 적용 추가 제출
bias_path = "[8.2]_best_bias.npy"
if os.path.exists(bias_path):
    try:
        best_bias = np.load(bias_path)
        test_probs_bias = apply_bias_to_probs(avg_test_probs, best_bias)
        sub_bias = pd.DataFrame({ID_COL: test_ids, TARGET:
test_probs_bias.argmax(axis=1).astype(int)})
        sub_bias.to_csv("[8.3]_submission_seed_ens_bias.csv", index=False)
        print(f"Bias 적용 제출 저장: [8.3]_submission_seed_ens_bias.csv (bias={best_bias})")
    except Exception as e:
        print(f"Bias 적용 실패 (무시하고 진행): {e}")

# 콘솔 요약
print("\n===== Seed Ensemble Summary =====")
for r in all_rows:
    print(f"seed={r['seed']} -> CV Macro F1: {r['mean_macro_f1']:.5f} ± {r['std_macro_f1']:.5f}")
print(f"\nAvg-OOF argmax Macro F1 (seed-ens): {overall_f1:.5f}")
print("Saved:",
      "[8.3]_cv_scores_seed_ensemble.csv",
      "[8.3]_oof_probs_seed_ens.npy",
      "[8.3]_oof_preds_seed_ens.npy",
      "[8.3]_test_probs_seed_ens.npy",
      "[8.3]_submission_seed_ens.csv",
      "(+ bias 제출이 있으면 [8.3]_submission_seed_ens_bias.csv)")
print(f"완료! 총 소요시간: {time.time()-t0:.1f}초")

if __name__ == "__main__":
    main()
```